```
// ===== FILE: app/android/app/src/main/java/io/flutter/plugins/GeneratedPluginRegistrant.java =====
package io.flutter.plugins;

import androidx.annotation.Keep;
import androidx.annotation.NonNull;
import io.flutter.Log;

import io.flutter.embedding.engine.FlutterEngine;

/**
 * Generated file. Do not edit.
 * This file is generated by the Flutter tool based on the
 * plugins that support the Android platform.
 */
@Keep
public final class GeneratedPluginRegistrant {
  private static final String TAG = "GeneratedPluginRegistrant";
  public static void registerWith(@NonNull FlutterEngine flutterEngine) {
    try {
      flutterEngine.getPlugins().add(new com.it_nomads.fluttersecurestorage.FlutterSecureStoragePlugin());
    } catch (Exception e) {
      Log.e(TAG, "Error registering plugin flutter_secure_storage, com.it_nomads.fluttersecurestorage.FlutterSecureStoragePlugin", e);
    }
    try {
      flutterEngine.getPlugins().add(new io.flutter.plugins.googlemobileads.GoogleMobileAdsPlugin());
    } catch (Exception e) {
      Log.e(TAG, "Error registering plugin google_mobile_ads, io.flutter.plugins.googlemobileads.GoogleMobileAdsPlugin", e);
    }
    try {
      flutterEngine.getPlugins().add(new com.github.dart_lang.jni.JniPlugin());
    } catch (Exception e) {
      Log.e(TAG, "Error registering plugin jni, com.github.dart_lang.jni.JniPlugin", e);
    }
    try {
      flutterEngine.getPlugins().add(new com.github.dart_lang.jni_flutter.JniFlutterPlugin());
    } catch (Exception e) {
      Log.e(TAG, "Error registering plugin jni_flutter, com.github.dart_lang.jni_flutter.JniFlutterPlugin", e);
    }
    try {
      flutterEngine.getPlugins().add(new io.flutter.plugins.sharedpreferences.SharedPreferencesPlugin());
    } catch (Exception e) {
      Log.e(TAG, "Error registering plugin shared_preferences_android, io.flutter.plugins.sharedpreferences.SharedPreferencesPlugin", e);
    }
    try {
      flutterEngine.getPlugins().add(new io.flutter.plugins.webviewflutter.WebViewFlutterPlugin());
    } catch (Exception e) {
      Log.e(TAG, "Error registering plugin webview_flutter_android, io.flutter.plugins.webviewflutter.WebViewFlutterPlugin", e);
    }
  }
}
```

```
// ===== FILE: app/android/app/src/main/kotlin/com/gamepouch/app/CheatStore.kt =====
package com.retroemupouch.app

import org.json.JSONArray
import org.json.JSONObject
import java.io.File

/** A selectable value inside a [ChoiceCheat] (e.g. 装备=剑 → its code). */
data class CheatOption(val label: String, val code: String)

/**
 * A user-facing cheat command. Commands are authored on the PC manager and
 * synced to the phone; the emulator never asks the player to type codes.
 */
sealed class CheatCommand {
    abstract var name: String

    /** The core code currently activated by this command, or null when off. */
    abstract val activeCode: String?

    abstract fun toJson(): JSONObject

    /** Simple on/off command, e.g. 「无限生命」. */
    class Switch(
        override var name: String,
        val code: String,
        var enabled: Boolean = false,
    ) : CheatCommand() {
        override val activeCode: String? get() = if (enabled) code else null
        override fun toJson() = JSONObject()
            .put("name", name).put("kind", "switch")
            .put("code", code).put("enabled", enabled)
    }

    /**
     * Value/equipment picker, e.g. 「武力值」= 10/50/100 or 「装备」= 剑/枪.
     * [selected] is the active option index, -1 = off (no code applied).
     */
    class Choice(
        override var name: String,
        val options: MutableList<CheatOption>,
        var selected: Int = -1,
    ) : CheatCommand() {
        override val activeCode: String?
            get() = options.getOrNull(selected)?.code

        override fun toJson() = JSONObject()
            .put("name", name).put("kind", "choice")
            .put("selected", selected)
```

```
                .put("options", JSONArray().apply {
                    options.forEach { put(JSONObject().put("label", it.label).put("code", it.code)) }
                })
        }

        companion object {
            fun fromJson(o: JSONObject): CheatCommand? {
                val name = o.optString("name").trim().ifEmpty { o.optString("code") }
                val kind = o.optString("kind").ifEmpty {
                    // Legacy entries (v1.2.0) were always simple switches.
                    if (o.has("code")) "switch" else ""
                }
                return when (kind) {
                    "choice" -> {
                        val arr = o.optJSONArray("options") ?: return null
                        val opts = (0 until arr.length()).mapNotNull { i ->
                            val p = arr.optJSONObject(i) ?: return@mapNotNull null
                            val code = p.optString("code").trim()
                            if (code.isEmpty()) return@mapNotNull null
                            CheatOption(
                                p.optString("label").trim().ifEmpty { code }, code)
                        }.toMutableList()
                        if (opts.isEmpty()) return null
                        val sel = o.optInt("selected", -1).let { if (it in opts.indices) it else -1 }
                        Choice(name, opts, sel)
                    }
                    "switch" -> {
                        val code = o.optString("code").trim()
                        if (code.isEmpty()) null
                        else Switch(name, code, o.optBoolean("enabled", false))
                    }
                    else -> null
                }
            }
        }
    }
}

/**
 * Per-game cheat command persistence.
 *
 * Storage:
 *   <filesDir>/library/cheats.json        — PC/ADB-synced or authored in-game
 *   <filesDir>/library/cloud_cheats.json  — LAN cloud sync snapshot
 *   both shaped { "version": 1, "byCrc": { "<CRC32-8HEX>": [ <command>, ... ] } }
 *
 * Commands are authored on the PC manager; the app only flips switches /
 * picks options and writes that state back. A non-empty local entry wins for
 * its CRC (it carries the player's toggle state); otherwise the cloud
 * snapshot is used — mirroring RomLibrary._applyCheats on the Dart side.
 */
```

```
object CheatStore {
    private const val DIR = "library"
    private const val FILE = "cheats.json"
    private const val CLOUD_FILE = "cloud_cheats.json"

    /** Codes that should currently be active for [commands]. */
    fun activeCodes(commands: List<CheatCommand>): List<String> =
        commands.mapNotNull { it.activeCode }

    /** Resolves the unsigned 8-hex CRC32 key for a ROM relative path. */
    fun crcKey(filesDir: File, romRel: String): String? {
        val idx = File(filesDir, "rom_index.json")
        if (!idx.exists()) return null
        return try {
            val arr = org.json.JSONArray(idx.readText())
            for (i in 0 until arr.length()) {
                val o = arr.optJSONObject(i) ?: continue
                if (o.optString("relativePath") == romRel) {
                    val v = o.optLong("crc32", 0L)
                    // Stored as a signed 32-bit int by the Dart side.
                    val u = if (v in 0..0xFFFFFFFFL) v else (v and 0xFFFFFFFFL)
                    return u.toString(16).padStart(8, '0').uppercase()
                }
            }
            null
        } catch (_: Exception) {
            null
        }
    }

    /** Loads commands for [romRel]; empty list when none / unreadable. */
    fun load(filesDir: File, romRel: String): MutableList<CheatCommand> {
        val key = crcKey(filesDir, romRel) ?: return mutableListOf()
        // Local (PC/ADB-synced or toggled in-game) wins when it actually has
        // commands; an empty/absent entry falls through to the cloud snapshot.
        readCommands(File(filesDir, "$DIR/$FILE"), key)?.let { if (it.isNotEmpty()) return it }
        readCommands(File(filesDir, "$DIR/$CLOUD_FILE"), key)?.let { return it }
        return mutableListOf()
    }

    /**
     * Parses the command array for [key] from one snapshot file.
     * Returns null when the file/key is absent or unreadable, an empty list
     * for a present-but-empty array.
     */
    private fun readCommands(f: File, key: String): MutableList<CheatCommand>? {
        if (!f.exists()) return null
        return try {
            val byCrc = JSONObject(f.readText()).optJSONObject("byCrc") ?: return null
            val arr = byCrc.optJSONArray(key) ?: return null
```

```
                (0 until arr.length()).mapNotNull { i ->
                    arr.optJSONObject(i)?.let(CheatCommand::fromJson)
                }.toMutableList()
            } catch (_: Exception) {
                null
            }
        }

        /** Replaces commands for [romRel] and atomically rewrites the store. */
        fun save(filesDir: File, romRel: String, commands: List<CheatCommand>) {
            val key = crcKey(filesDir, romRel) ?: return
            val dir = File(filesDir, DIR).apply { mkdirs() }
            val f = File(dir, FILE)
            val root = if (f.exists()) {
                try { JSONObject(f.readText()) } catch (_: Exception) { JSONObject() }
            } else JSONObject()
            var byCrc = root.optJSONObject("byCrc")
            if (byCrc == null) {
                byCrc = JSONObject()
                root.put("version", 1)
                root.put("byCrc", byCrc)
            }
            val arr = JSONArray()
            commands.forEach { arr.put(it.toJson()) }
            byCrc.put(key, arr)
            val tmp = File(dir, "$FILE.tmp")
            tmp.writeText(root.toString(2))
            tmp.renameTo(f)
        }
    }

    // ===== FILE: app/android/app/src/main/kotlin/com/gamepouch/app/EmulatorActivity.kt =====
    package com.retroemupouch.app

    import android.app.Activity
    import android.app.AlertDialog
    import android.content.Context
    import android.content.Intent
    import android.content.res.Configuration
    import android.graphics.Bitmap
    import android.graphics.BitmapFactory
    import android.graphics.Color
    import android.graphics.drawable.GradientDrawable
    import android.graphics.drawable.StateListDrawable
    import android.graphics.drawable.Drawable
    import android.opengl.GLES20
    import android.opengl.GLSurfaceView
    import android.os.Bundle
    import android.util.Log
    import android.view.Gravity
```

```
import android.view.View
import android.view.ViewGroup
import android.view.ViewOutlineProvider
import android.view.WindowManager
import android.widget.Button
import android.widget.CheckBox
import android.widget.FrameLayout
import android.widget.ImageView
import android.widget.LinearLayout
import android.widget.ScrollView
import android.widget.TextView
import android.widget.Toast
import com.retroemupouch.app.netplay.NetplaySessionCfg
import com.retroemupouch.app.netplay.NpLog
import com.retroemupouch.app.netplay.PlayBootstrapClient
import com.retroemupouch.app.netplay.PlayBootstrapClient.SramResult
import com.retroemupouch.app.netplay.PlayBootstrapServer
import com.retroemupouch.app.netplay.atomicWriteWithBackup
import io.flutter.embedding.engine.FlutterEngineCache
import io.flutter.plugin.common.MethodChannel
import java.io.File
import java.nio.ByteBuffer
import java.nio.ByteOrder
import java.text.SimpleDateFormat
import java.util.Calendar
import java.util.Locale
import java.util.concurrent.CountDownLatch
import java.util.concurrent.Executors
import java.util.concurrent.TimeUnit
import java.util.zip.CRC32
import javax.microedition.khronos.egl.EGLConfig
import javax.microedition.khronos.opengles.GL10

/**
 * Screen-relative layout helper. Converts percentage values (0..1) to
 * pixel values based on the device's current screen dimensions.
 *
 * Used by [EmulatorActivity] (and any future view that wants the same
 * behaviour) so the in-game UI scales identically across phones — a 5"
 * device, the HBN-AL00 reference, or a tablet all render the same
 * proportions without per-device tuning. Captured once in
 * [EmulatorActivity.onCreate] from `resources.displayMetrics`.
 *
 * Conventions:
 *  - [wp] horizontal dimensions (margins, gaps, button widths).
 *  - [hp] vertical dimensions (top margins, panel paddings, button
 *    heights).
 *  - [sp] control sizes (icons, knobs) — the shorter of width/height so
 *    a landscape phone and a portrait phone get the same visual size.
 *
```

```
     * Every input is expected in 0..1; values outside that range are passed
     * through as-is, so callers can also express "0.5dp above safeTopPx" by
     * mixing fractions and pixel offsets at the call site.
     */
    private class Sizing(wPx: Int, hPx: Int) {
        /** Current screen width in raw pixels. */
        val w: Float = wPx.toFloat()
        /** Current screen height in raw pixels. */
        val h: Float = hPx.toFloat()
        /** Shorter of [w] / [h] — the "short side" used for control sizing. */
        val min: Float = minOf(w, h)

        /** Pixel value equal to [pct] (0..1) of the current screen width. */
        fun wp(pct: Float): Int = (w * pct + 0.5f).toInt()
        /** Pixel value equal to [pct] (0..1) of the current screen height. */
        fun hp(pct: Float): Int = (h * pct + 0.5f).toInt()
        /** Pixel value equal to [pct] (0..1) of the shorter screen dimension. */
        fun sp(pct: Float): Int = (min * pct + 0.5f).toInt()
    }

    /**
     * Full-screen native emulation screen.
     *
     * Pacing: a dedicated high-priority native thread steps the core on an absolute
     * CLOCK_MONOTONIC timeline at the libretro AV rate (NES NTSC ~60.0988 fps),
     * independent of the GL/display refresh (60/90/120 Hz) so a 120 Hz panel does
     * not run the game at 2x speed and GL-driver jitter cannot drop an emulated
     * frame. The GL thread only pumps the latest published frame and presents it.
     *
     * Benchmark mode (bench_* intent extras): virtual pad is scripted, overlay hidden;
     * after warm-up + sample windows the runner writes filesDir/bench/<id>.json and the
     * activity finishes automatically. logcat is unavailable on the HarmonyOS 4.2 AOSP
     * compat layer, so all evidence lives in files (bench/ + logs/).
     */
    class EmulatorActivity : Activity() {

        private lateinit var glView: GLSurfaceView
        private lateinit var pad: VirtualPadView
        private lateinit var hud: TextView
        private var handle = 0L
        private var benchMode = false
        // Set by 重开: recreate() pauses the old instance first, whose onPause
        // must not re-snapshot the state we are about to delete.
        private var restarting = false
        // Decrypted ROM cache file (lives in filesDir/rom_cache). Non-null
        // only when the launched ROM was an MGS1-locked ZIP. Deleted in
        // onDestroy so the plaintext is gone as soon as the activity tears
        // down.
        private var plainRom: java.io.File? = null
        private var speedButton: Button? = null
```

```
    private val speedSteps = intArrayOf(1, 2, 4, 6, 8)
    private var speedIndex = 0
    // Corner toolbar (≡ top-left, ✕ top-right, expanding pill panel).
    // Stored as fields so the WindowInsets listener (see onCreate) can
    // re-anchor them below the status bar — a fixed 6dp topMargin sinks
    // them behind the system bar on phones with a tall bar or punch-hole
    // (e.g. Xiaomi / MIUI).
    private var menuIconLp: FrameLayout.LayoutParams? = null
    private var exitIconLp: FrameLayout.LayoutParams? = null
    private var menuPanelLp: FrameLayout.LayoutParams? = null
    private var menuIcon: View? = null
    private var exitIcon: View? = null
    private var menuPanel: View? = null

    // Root view + deferred-chrome bookkeeping. The corner toolbar is built
    // only after the first real layout pass (not from onCreate's
    // displayMetrics): Android VM containers (Zhuoyitong on HarmonyOS NEXT,
    // etc.) may report portrait metrics at create time and rotate the
    // window afterwards, while configChanges means onCreate never re-runs.
    // Sizing from the laid-out view — plus re-flow on size changes — keeps
    // the chrome anchored in every container rotation.
    private lateinit var root: FrameLayout
    private var chromeInstalled = false
    private var chromeW = 0
    private var chromeH = 0
    private var chromeInsetTop = 0

    // ------------------------------------------------------------ netplay
    private var npCfg: NetplaySessionCfg? = null
    private var npServer: PlayBootstrapServer? = null
    private var npClient: PlayBootstrapClient? = null
    private var npOverlay: TextView? = null
    private var npStat: TextView? = null
    private var npWifiLock: android.net.wifi.WifiManager.WifiLock? = null
    private val npExecutor = Executors.newSingleThreadExecutor { r ->
        Thread(r, "np-ctl").apply { isDaemon = true }
    }
    /** Guest counts this after the pre-init SRAM sync; host counts it at once. */
    private val npGateInit = CountDownLatch(1)
    /** GL thread counts this after core init + the geometry probe frame. */
    private val npCoreReady = CountDownLatch(1)
    @Volatile private var npEnded = false
    @Volatile private var npRunning = false
    @Volatile private var npPaused = false
    private var npPausedTotalMs = 0L
    private var npPausedSinceMs = 0L

    // Netplay-safe one-button special moves: the macro is replayed frame by
    // frame into the SAME local-mask channel the pad uses (renderer.buttonMask,
    // pushed per frame via nativeNetplayLocalMask), so the sequence is just
```

```
                // ordinary local input and cannot desync the pair — unlike
    // nativePlayMacro, which bypasses the per-frame bitmask.
    @Volatile private var npPadMask = 0
    @Volatile private var npMacroMask = 0
    private val npMacroHandler = android.os.Handler(android.os.Looper.getMainLooper())
    private var npMacroPending: Runnable? = null
    // Wall-clock start of the peer's current negotiated pause (0 = not
    // paused); drives the 15 s remote-pause watchdog in onNetplayStats.
    private var npRemotePausedSinceMs = 0L
    private var npLastFrames = -1L
    private var npStallPolls = 0
    private var npDroppedHandled = false
    private var npFailFinished = false
    // Bootstrap failed before the core was created: release the GL gate so it
    // does not burn its own 40 s timeout and stack a second error dialog.
    private val npBootAborted = java.util.concurrent.atomic.AtomicBoolean(false)
    // Guest gate wait runs from the frame loop (never inside
    // onSurfaceCreated): buffers swapped during that callback are not shown
    // and the system keeps the white starting window up, so the parked guest
    // would stare at a blank white screen for the whole SRAM sync.
    @Volatile private var npBootDeferred = false
    @Volatile private var npBootDeadlineNs = 0L

    /**
     * Screen-relative layout helper for the corner toolbar / panel /
     * HUD. Initialised at the top of [onCreate] from
     * `resources.displayMetrics` so every dimension (icon size, margin,
     * panel button size, HUD gap) is a fraction of the screen and stays
     * in proportion on every phone — no per-device tuning required.
     */
    private lateinit var sizing: Sizing

    // Cheat commands (null in bench mode / when the ROM has no index entry).
    private var romRelPath: String? = null
    private var cheats: MutableList<CheatCommand>? = null
    private val cheatLock = Any()
    // Per-game storage key (CRC32 hex when indexed, else ROM file base).
    private lateinit var saveKey: String

    private val renderer = EmuRenderer()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // The theme (shared NormalTheme) is light and paints a white window;
        // the emulator surface is black, so never flash white behind it.
        window.setBackgroundDrawable(android.graphics.drawable.ColorDrawable(Color.BLACK))
        window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)

        // Capture screen dimensions *once* per onCreate. Resources.displayMetrics
        // is updated by Android on rotation / fold / multi-window before onCreate
```

```
        // fires, so the values already reflect the current orientation; every
        // later percentage (icon size, corner margin, panel button, HUD gap, …)
        // is a fraction of these pixels and stays in proportion on every phone.
        sizing = Sizing(
            resources.displayMetrics.widthPixels,
            resources.displayMetrics.heightPixels
        )

        val romExtra = intent.getStringExtra("rom")
        val benchRom = intent.getStringExtra("bench_rom")
        benchMode = benchRom != null
        val romRel = benchRom ?: romExtra
        if (romRel == null) { finish(); return }
        val romFile = File(filesDir, romRel)
        // Locked-ROM path: the on-disk file is a single-entry .zip whose
        // payload is the MGS1-encrypted ROM. Materialise the plaintext in
        // the app-private cache and pass *that* to the native runner; we
        // never expose the cipher bytes or the decrypted bytes past the
        // session. `plainRom` is deleted in onDestroy.
        plainRom = if (romFile.name.endsWith(".zip", ignoreCase = true) ||
                       isLikelyMgs1(romFile) ||
                       isLikelyZip(romFile)) {
            val coreForLaunch = (intent.getStringExtra("core")
                ?: intent.getStringExtra("bench_core"))
                ?.takeIf { it.isNotBlank() } ?: "fceumm"
            try { LockedRom.materialize(filesDir, romFile, coreForLaunch) } catch (e: Throwable) {
                Log.e("GS-Rom", "decrypt failed: ${e.message}"); finish(); return
            }
        } else null
        val saveDir = File(filesDir, "saves").apply { mkdirs() }.absolutePath
        val sysDir = File(filesDir, "system").apply { mkdirs() }.absolutePath
        val logDir = File(filesDir, "logs").apply { mkdirs() }
        val logPath = File(logDir, "run-${System.currentTimeMillis()}.log").absolutePath
        NpLog.init(logDir)
        NpLog.log("EmulatorActivity onCreate pid=${android.os.Process.myPid()}")

        glView = GLSurfaceView(this).apply {
            setEGLContextClientVersion(2)
            // Keep the EGL context across background/foreground when the
            // driver allows it; the renderer also tolerates full recreation
            // (core state lives in the native runner, not in GL objects).
            preserveEGLContextOnPause = true
            setRenderer(renderer)
            renderMode = GLSurfaceView.RENDERMODE_CONTINUOUSLY
        }
        // Battery saves and quick-save states are keyed by the ROM's stable
        // CRC32 so different games can never share a save file; fall back to
        // the file base name for unindexed ROMs (benchmark / sideloaded).
        val crcKey = CheatStore.crcKey(filesDir, romRel)
        val baseName = romFile.nameWithoutExtension
```

```
                saveKey = crcKey ?: baseName
        // Netplay sessions always key saves by the signaling CRC. A malformed
        // config is rejected before touching the runner.
        npCfg = NetplaySessionCfg.from(intent)
        if (npCfg == null) {
            NpLog.log("netplay cfg absent (intent has np_* extras? " +
                "role=${intent.getIntExtra("np_role", -1)} " +
                "peer=${intent.getStringExtra("np_peer_ip")}:" +
                "${intent.getIntExtra("np_bootstrap_port", -1)} " +
                "room=${intent.getIntExtra("np_room_id", 0)})")
        } else {
            NpLog.log("netplay cfg role=${npCfg!!.role} peer=${npCfg!!.peerIp} " +
                "peerPort=${npCfg!!.peerUdpPort} localPort=${npCfg!!.localUdpPort} " +
                "bootPort=${npCfg!!.bootstrapPort} room=${npCfg!!.roomId} " +
                "D=${npCfg!!.initialD} keyLen=${npCfg!!.sessionKey.length} save=${npCfg!!.saveKey}")
        }
        npCfg?.let { saveKey = it.saveKey }
        runningSaveKey = saveKey
        // When the input is locked, point the runner at the plaintext
        // materialised above; otherwise at the file as-is. Saves / quick-
        // state still key off the locked file's name so they stay valid
        // across launches.
        val runnerRom = plainRom?.absolutePath ?: romFile.absolutePath
        renderer.romPath = runnerRom
        renderer.saveDir = saveDir
        renderer.systemDir = sysDir
        renderer.logPath = logPath
        renderer.saveKey = saveKey
        renderer.coreId = (intent.getStringExtra("core")
            ?: intent.getStringExtra("bench_core"))
            ?.takeIf { it.isNotBlank() } ?: "fceumm"
        renderer.benchSeconds = intent.getIntExtra("bench_seconds", 0)
        renderer.netCfg = npCfg
        // Load local / cloud-synced cheats before the GL thread inits the
        // core; the renderer seeds them right after retro_load_game().
        // Netplay forbids cheats entirely (determinism): the list stays null.
        if (!benchMode && npCfg == null) {
            romRelPath = romRel
            cheats = CheatStore.load(filesDir, romRel)
        }
        // Safe insets are derived from the surface size in onSurfaceChanged():
        // every platform plays on the same reserved "stage" (at least the NES
        // 8:7 side bars, enlarged where a profile's controls need more room),
        // so the picture never slides under the pad / corner menu. Benchmark
        // mode keeps a full-screen surface.
        renderer.onBenchDone = { finish() }
        renderer.onFatal = { msg -> runOnUiThread {
            // A plain Toast disappears before the user can read it (the
            // activity finishes immediately), which reads as a silent crash.
            // Keep a modal dialog up so the concrete reason is visible.
```

```
                AlertDialog.Builder(this)
                    .setTitle("游戏无法启动")
                    .setMessage(msg)
                    .setCancelable(false)
                    .setPositiveButton("返回文件库") { _, _ -> finish() }
                    .show()
            } }

            root = FrameLayout(this)
            // Black backdrop: while the guest parks the GL thread on the
            // bootstrap gate, no frame can be drawn — without this the window
            // shows the default white background behind "正在同步存档…".
            root.setBackgroundColor(Color.BLACK)
            root.addView(glView, FrameLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.MATCH_PARENT
            ))

            hud = TextView(this).apply {
                setTextColor(Color.YELLOW)
                textSize = 10.5f
                setLineSpacing(2f, 1f)
                setBackgroundColor(Color.argb(90, 0, 0, 0))
                setPadding(10, 6, 10, 6)
                visibility = if (benchMode) View.INVISIBLE else View.VISIBLE
            }
            root.addView(hud, FrameLayout.LayoutParams(
                ViewGroup.LayoutParams.WRAP_CONTENT, ViewGroup.LayoutParams.WRAP_CONTENT,
                Gravity.TOP or Gravity.START
            ).apply {
                // Initial placement is short-lived — [layoutHud] re-anchors
                // against safeTopPx after the first GL frame arrives. Small
                // percentages keep the HUD inside the play field until then
                // without relying on per-device tuning.
                topMargin = sizing.hp(0.04f)
                marginStart = sizing.wp(0.04f)
            })

            if (!benchMode) {
                pad = VirtualPadView(this)
                // Load the user-editable fight-assist palette (波/升/旋/超 …)
                // before any layout pass, so the very first draw already uses
                // the configured scripts. Falls back to the KOF97 defaults
                // shipped in `assets/moves_presets.json` when the on-disk
                // `library/moves.json` is missing / corrupt, so a brand-new
                // install still renders the familiar palette.
                try {
                    val cfg = SpecialMovesStore.load(filesDir, applicationContext)
                    pad.setMacros(cfg.activeProfile().moves.map {
                        it.label to runCatching { ScriptCompiler.compile(it.script) }
                            .getOrElse { IntArray(0) }
```

```
                })
                Log.i("GS-Emu",
                    "moves loaded active=${cfg.activeProfileId} " +
                    "moves=${cfg.activeProfile().moves.size}")
            } catch (t: Throwable) {
                Log.w("GS-Emu", "moves load failed: ${t.message}")
            }
            pad.profile = VirtualPadView.Profile.from(
                intent.getStringExtra("core"),
                intent.getStringExtra("platform"),
                romFile.name)
            // Netplay: the driven port is fixed by the session role, so the
            // arcade 1P/2P toggle is hidden; the mask still routes through
            // renderer.buttonMask but is pushed via nativeNetplayLocalMask.
            if (npCfg != null) {
                pad.portLocked = true
                // Coin-op guest sees the same picture at once but only spawns
                // as 2P after coin+START — surface that instead of letting it
                // look like the second pad "does nothing".
                if (npCfg!!.role == 1 &&
                    (pad.profile == VirtualPadView.Profile.MAME ||
                     pad.profile == VirtualPadView.Profile.NEOGEO)) {
                    pad.joinHint = "你是 2P：右上「菜单」→ 投币 → 开始 加入游戏"
                }
            }
            pad.listener = VirtualPadView.OnButtonsChanged { mask, port ->
                npPadMask = mask
                renderer.buttonMask = mask or npMacroMask
                renderer.playerPort = port
            }
            // One-button special moves (fighting games). The compiled per-frame
            // mask sequence is replayed through the SAME 50 ms/step channel the
            // on-screen pad uses (renderer.buttonMask, OR-ed over the live pad),
            // for BOTH solo and netplay. This mirrors the HarmonyOS JS replay
            // exactly: it releases on every step change and emits a final release
            // frame, so combos latch AND the buttons always let go — unlike the
            // old native nativePlayMacro path, which held the last frame and left
            // the input stuck ("组合按键释放不出来"). Netplay stays desync-free
            // because each step is just ordinary local input the lockstep gate
            // already transmits.
            pad.macroListener = object : VirtualPadView.OnMacro {
                override fun onMacro(seq: IntArray) {
                    if (handle == 0L) return
                    startNpMacro(seq)
                }
                override fun onClear() {
                    cancelNpMacro()
                }
            }
            root.addView(pad, FrameLayout.LayoutParams(
```

```
            ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.MATCH_PARENT
        ))
        // Corner toolbar (功能/✕/pop-out panel) is installed lazily by
        // installChrome() once the root view reports real pixels — see
        // the layout listener registered after setContentView.
    } else {
        pad = VirtualPadView(this) // unused, keeps renderer contract simple
    }

    // Netplay phase overlay + live session HUD.
    npCfg?.let { cfg ->
        npOverlay = TextView(this).apply {
            text = "正在建立联机…"
            setTextColor(Color.WHITE)
            textSize = 16f
            gravity = Gravity.CENTER
            setBackgroundColor(Color.argb(210, 0, 0, 0))
            visibility = View.VISIBLE
        }
        root.addView(npOverlay, FrameLayout.LayoutParams(
            ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.MATCH_PARENT))
        npStat = TextView(this).apply {
            setTextColor(Color.WHITE)
            textSize = 11f
            setBackgroundColor(Color.argb(120, 0, 0, 0))
            setPadding(14, 5, 14, 5)
            visibility = View.GONE
        }
        root.addView(npStat, FrameLayout.LayoutParams(
            ViewGroup.LayoutParams.WRAP_CONTENT, ViewGroup.LayoutParams.WRAP_CONTENT,
            Gravity.TOP or Gravity.CENTER_HORIZONTAL
        ).apply { topMargin = sizing.hp(0.045f).toInt() })
        acquireNetplayWifiLock()
        // Host's core boot has no precondition; the guest waits until its
        // SRAM pull finishes so the core loads the host's battery save.
        if (cfg.isHost) npGateInit.countDown()
        NpLog.log("np-ctl dispatch isHost=${cfg.isHost}")
        npExecutor.execute {
            try {
                if (cfg.isHost) hostBootstrap(cfg) else guestBootstrap(cfg)
            } catch (t: Throwable) {
                NpLog.log("bootstrap FAILED ${t.javaClass.name}: ${t.message} | " +
                    (t.stackTrace.take(3).joinToString(" < ") {
                        "${it.fileName}:${it.lineNumber} ${it.methodName}" }))
                Log.e("GS-Netplay", "bootstrap failed", t)
                netplayFail("联机启动失败: ${t.message ?: "网络错误"}")
            }
        }
    }
```

```
renderer.onStageChanged = { runOnUiThread { layoutHud() } }

        // ----- apply persisted settings (see AppSettings on the Dart side) -----
        applyPersistedSettings()

        setContentView(root)
        hud.post { layoutHud() }
        preferHighRefreshMode()

        // Re-anchor the corner toolbar under the system status bar /
        // display cutout. On Honor with a translucent bar a fixed 6dp
        // margin is fine; on Xiaomi/MIUI the bar is opaque and taller
        // (≈96px on most 2025-era Redmi/K phones, more on devices with
        // a centred punch-hole), which would hide the ≡ / ✕ icons.
        // The listener survives rotation and is also the source of truth
        // when a VM container reports its insets late; applyChromeTopInset
        // re-anchors the lazily-installed toolbar (or just remembers the
        // value until installChrome runs).
        root.setOnApplyWindowInsetsListener { _, insets ->
            // Chrome row only needs the top (status bar / cutout).
            val statusBar = insets.getInsets(
                android.view.WindowInsets.Type.statusBars()
                    or android.view.WindowInsets.Type.displayCutout()
            )
            applyChromeTopInset(statusBar.top)
            // The GL stage needs every edge: on edge-to-edge layouts
            // (Android 15, Zhuoyitong on HarmonyOS NEXT) the surface runs
            // under the floating status/navigation bars and the picture
            // would otherwise be painted beneath them (looks cropped).
            if (!benchMode) {
                val bars = insets.getInsets(
                    android.view.WindowInsets.Type.statusBars()
                        or android.view.WindowInsets.Type.navigationBars()
                        or android.view.WindowInsets.Type.displayCutout()
                )
                renderer.setSystemInsets(bars.left, bars.top, bars.right, bars.bottom)
            }
            // Don't consume — let GL surface still get insets if it wants.
            insets
        }

        // Install the corner toolbar from the *real laid-out pixels* (not
        // onCreate's displayMetrics), and re-flow it whenever the window
        // changes size — covers VM containers that create the activity in
        // portrait metrics and rotate it into landscape afterwards without
        // recreating it (configChanges).
        root.addOnLayoutChangeListener { _, _, _, _, _, _, _, _, _ ->
            if (benchMode) return@addOnLayoutChangeListener
            val w = root.width
            val h = root.height
```

```
            if (w <= 0 || h <= 0) return@addOnLayoutChangeListener
            root.post {
                if (!chromeInstalled) installChrome(w, h)
                else onChromeSizeChanged(w, h)
            }
        }
        // Initial dispatch in case the view tree was already laid out
        // before the listener was attached (some OEMs fire insets twice).
        root.requestApplyInsets()
    }

    override fun onConfigurationChanged(newConfig: Configuration) {
        super.onConfigurationChanged(newConfig)
        // configChanges keeps this activity alive across the container's
        // rotation; the layout listener normally re-flows everything, give
        // it one more nudge after resources have settled.
        if (::root.isInitialized) root.post {
            val w = root.width
            val h = root.height
            if (w > 0 && h > 0) {
                if (!chromeInstalled) installChrome(w, h)
                else onChromeSizeChanged(w, h)
            }
        }
    }

    /**
     * Build and attach the corner toolbar (功能 opener · ✕ exit · pop-out
     * panel) sized from the window's real laid-out pixels [w]×[h]. Runs
     * exactly once, on the first valid layout pass. Pill pixel sizes only
     * depend on the short side, which is identical before/after a
     * portrait<->landscape swap of a fullscreen window, so later swaps are
     * handled by [onChromeSizeChanged] without rebuilding the pills.
     */
    private fun installChrome(w: Int, h: Int) {
        if (chromeInstalled || benchMode || w <= 0 || h <= 0) return
        chromeInstalled = true
        chromeW = w
        chromeH = h
        sizing = Sizing(w, h)

        val menu = buildMenuBar(sizing, netplay = npCfg != null)
        val marginV  = sizing.hp(0.017f)   // ~6dp on 360dp-tall
        val marginH  = sizing.wp(0.017f)   // ~6dp on 360dp-wide
        val panelGap = sizing.wp(0.012f)   // matches pad inter-pill gap
        val pillW    = sizing.sp(0.1988f)
        val pillH    = sizing.sp(0.077f)
        // Pills are shorter than the legacy circular icon; nudge the whole
        // top row down by half the difference so the row stays centred on
        // the same band it used to occupy.
```

```
val rowNudge = (sizing.sp(0.10f) - pillH) / 2

        // Top-left: "功能" opener pill.
        val menuLp = FrameLayout.LayoutParams(
            pillW, pillH, Gravity.TOP or Gravity.START
        ).apply { topMargin = marginV + rowNudge; marginStart = marginH }
        root.addView(menu.first, menuLp)

        // Top-right: compact round exit "✕".
        val exitLp = FrameLayout.LayoutParams(
            pillH, pillH, Gravity.TOP or Gravity.END
        ).apply { topMargin = marginV + rowNudge; marginEnd = marginH }
        root.addView(menu.second, exitLp)

        // Pop-out panel level with the opener, extending to its right.
        menuPanelLp = FrameLayout.LayoutParams(
            ViewGroup.LayoutParams.WRAP_CONTENT, ViewGroup.LayoutParams.WRAP_CONTENT,
            Gravity.TOP or Gravity.START
        ).apply {
            topMargin = marginV + rowNudge
            // Subtract the first pill's own start margin (wp 0.006) so the
            // opener->first-pill gap equals one panelGap, exactly like the
            // pad's expanded row.
            marginStart = marginH + pillW + panelGap - sizing.wp(0.006f)
        }
        root.addView(menu.third, menuPanelLp)

        menuIconLp = menuLp
        exitIconLp = exitLp
        menuIcon = menu.first
        exitIcon = menu.second
        menuPanel = menu.third

        applyChromeTopInset(chromeInsetTop)
        // Netplay live-stats row uses a height-fraction top margin too.
        npStat?.let {
            val lp = it.layoutParams as FrameLayout.LayoutParams
            lp.topMargin = sizing.hp(0.045f)
            it.layoutParams = lp
        }
        root.requestApplyInsets()
        layoutHud()
    }

    /**
     * Re-flow toolbar margins after an orientation / window-size swap.
     * Short-side is unchanged by a 90° swap, so pill sizes stay valid;
     * only width/height-fraction margins and the inter-pill gaps move.
     */
    private fun onChromeSizeChanged(w: Int, h: Int) {
```

```
        if (!chromeInstalled || w <= 0 || h <= 0) return
        if (w == chromeW && h == chromeH) return
        chromeW = w
        chromeH = h
        sizing = Sizing(w, h)

        val marginH  = sizing.wp(0.017f)
        val panelGap = sizing.wp(0.012f)
        val pillW    = sizing.sp(0.1988f)
        menuIconLp?.let {
            it.marginStart = marginH
            menuIcon?.layoutParams = it
        }
        exitIconLp?.let {
            it.marginEnd = marginH
            exitIcon?.layoutParams = it
        }
        menuPanelLp?.let {
            it.marginStart = marginH + pillW + panelGap - sizing.wp(0.006f)
            menuPanel?.layoutParams = it
        }
        // Inter-pill gaps inside the pop-out panel are width fractions.
        (menuPanel as? LinearLayout)?.let { ll ->
            for (i in 0 until ll.childCount) {
                (ll.getChildAt(i).layoutParams as? LinearLayout.LayoutParams)?.let { clp ->
                    clp.marginStart = sizing.wp(0.006f)
                    clp.marginEnd = sizing.wp(0.006f)
                }
            }
        }
        applyChromeTopInset(chromeInsetTop)
        npStat?.let {
            val lp = it.layoutParams as FrameLayout.LayoutParams
            lp.topMargin = sizing.hp(0.045f)
            it.layoutParams = lp
        }
        layoutHud()
    }

    /**
     * Push the chrome row's bottom edge (window coordinates) to the virtual
     * pad so its R/L / SELECT / START stack clears the 功能/✕ row on every
     * device — including VM containers that don't push the full-screen pad
     * view down by the status-bar inset.
     */
    private fun notifyPadChromeBottom() {
        if (benchMode || !chromeInstalled || !::pad.isInitialized) return
        val top = chromeInsetTop + sizing.hp(0.017f) +
            (sizing.sp(0.10f) - sizing.sp(0.077f)) / 2f
        // root is fullscreen at window-Y 0, so local Y == window Y here.
```

```
        pad.setTopChromeBottom(top + sizing.sp(0.077f))
    }

    /**
     * Sit every corner control [insetTop] pixels below the system bar /
     * display cutout (plus the screen-relative 1.7% floor). Remembers the
     * latest inset even before the toolbar is installed.
     */
    private fun applyChromeTopInset(insetTop: Int) {
        chromeInsetTop = insetTop
        if (!chromeInstalled) return
        val topMargin = insetTop + sizing.hp(0.017f) +
            (sizing.sp(0.10f) - sizing.sp(0.077f)) / 2
        menuIconLp?.let { it.topMargin = topMargin; menuIcon?.layoutParams = it }
        exitIconLp?.let { it.topMargin = topMargin; exitIcon?.layoutParams = it }
        menuPanelLp?.let { it.topMargin = topMargin; menuPanel?.layoutParams = it }
        notifyPadChromeBottom()
    }

    /**
     * Read settings the user picked in the Flutter Settings sheet (which
     * persists via shared_preferences) and reflect them in this native
     * activity. SharedPreferences from `flutter` and Kotlin share the
     * same backing file under the standard app user data directory.
     *
     * Modern shared_preferences (≥ 2.x) writes its entries to a file
     * named `FlutterSharedPreferences.xml` under the app's shared_prefs
     * directory; older releases used `flutter.0.xml` / `flutter.1.xml`.
     * We open the modern file directly to stay compatible with the
     * version bundled in pubspec.
     */
    private fun applyPersistedSettings() {
        val prefs = getSharedPreferences("FlutterSharedPreferences", MODE_PRIVATE)
        // FPS HUD overlay — visibility is the single source of truth.
        val showHud = prefs.getBoolean("flutter.showFpsHud", true)
        if (::hud.isInitialized) hud.visibility = if (showHud) View.VISIBLE else View.GONE

        // Virtual pad mode (stickMode / deadzone / useDpadUi) — only
        // applied when the pad is in use; the bench-mode placeholder has
        // no surface to update. Read stickDeadzone as String (NOT getFloat)
        // because `shared_preferences.setDouble` writes base64-encoded bytes
        // which throw ClassCastException on getFloat — the Flutter app writes
        // the human-readable form ("0.18") via setString + toString().
        if (::pad.isInitialized) {
            val mode = prefs.getString("flutter.stickMode", "DPAD8")
            pad.stickMode = when (mode) {
                "STICK" -> VirtualPadView.StickMode.STICK
                "DPAD4" -> VirtualPadView.StickMode.DPAD4
                else -> VirtualPadView.StickMode.DPAD8
            }
```

```
            val dz = prefs.getString("flutter.stickDeadzone", "0.18")
                ?.toFloatOrNull() ?: 0.18f
            pad.deadzone = dz
            pad.useDpadUi = prefs.getBoolean("flutter.useDpadUi", false)
            // 用户可配置布局：整体缩放(默认 1.0) + 整体平移(相对屏幕宽/高的比例)。
            pad.padScale = prefs.getString("flutter.padScale", "1.0")
                ?.toFloatOrNull()?.coerceIn(0.5f, 2.5f) ?: 1.0f
            pad.padOffsetX = prefs.getString("flutter.padOffsetX", "0.0")
                ?.toFloatOrNull()?.coerceIn(-0.45f, 0.45f) ?: 0.0f
            pad.padOffsetY = prefs.getString("flutter.padOffsetY", "0.0")
                ?.toFloatOrNull()?.coerceIn(-0.45f, 0.45f) ?: 0.0f
            // 布局变化需要重算几何，否则下一帧 onSizeChanged 才会生效。
            pad.requestLayout()
        }

        // "Fast-forward audio when ×N" — when off, we override the per-
        // frame audio rate so the audio buffer is silenced during fast-
        // forward; when on, audio speeds up proportionally with the
        // emulation rate (current default behaviour). We mark it here so
        // cycleSpeed() can check the pref when toggling the rate.
        fastForwardAudioPref = prefs.getBoolean("flutter.fastForwardAudio", true)

        // "Remember last game" — restore the last save slot the user was
        // playing so 「继续游戏」 lands on the exact last frame.
        rememberLastGamePref = prefs.getBoolean("flutter.rememberLastGame", true)
        if (rememberLastGamePref) {
            val slot = prefs.getInt("flutter.lastSaveSlot", -1)
            if (slot in 0..SLOT_COUNT - 1) {
                val f = slotStateFile(slot)
                if (f.exists()) renderer.loadSlot(f)
            }
        }
    }

    /**
     * In-memory mirror of the Flutter-side preferences. Mutated by
     * [applyPersistedSettings] at activity create; consulted by the
     * menu actions (cycleSpeed, saveToSlot, …) when they need to honour
     * the user's settings live.
     */
    private var fastForwardAudioPref: Boolean = true
    private var rememberLastGamePref: Boolean = true

    /**
     * Place the FPS/xruns readout in the free side bar, hugging the game
     * frame's left edge. It sits two rows ([HUD_LIFT_DP]) above vertical
     * centre, so it never overlaps the L shoulder or the stick; clamped to
     * stay below the top-left menu bar. In portrait (no side bar) it falls
     * back to the top strip under the shoulder row.
     */
```

```
    private fun layoutHud() {
        if (!::hud.isInitialized || hud.visibility != View.VISIBLE) return
        hud.post {
            val parentW = (hud.parent as? View)?.width ?: 0
            val parentH = (hud.parent as? View)?.height ?: 0
            if (hud.measuredWidth <= 0 || parentW <= 0 || parentH <= 0) return@post
            val lp = hud.layoutParams as FrameLayout.LayoutParams
            lp.gravity = Gravity.TOP or Gravity.START
            // Top-center of the picture area. We sit a screen-relative
            // fraction below safeTopPx (the top of the picture) so the
            // readout doesn't hug the status-bar inset; we horizontally
            // centre on the picture axis so it stays clear of both
            // corner buttons. Reading is a quick glance up rather than
            // obstructing the playfield.
            val picCx = (parentW + renderer.safeLeftPx - renderer.safeRightPx) / 2f
            lp.topMargin = (renderer.safeTopPx + sizing.hp(0.011f)).toInt()
            lp.marginStart = (picCx - hud.measuredWidth / 2f).toInt()
            hud.layoutParams = lp
        }
    }

    /**
     * Ask the panel for its highest refresh mode (120 Hz when available). The
     * emulated Dendy/PAL core produces 50 fps: at 60 Hz each frame is held for
     * 1 or 2 vsyncs (16.7/33.3 ms) while at 120 Hz it is held for 2 or 3
     * (16.7/25 ms), so the duplicated-frame judder during horizontal scroll is
     * roughly halved. Falls back silently when the platform refuses the mode.
     */
    @Suppress("DEPRECATION")
    private fun preferHighRefreshMode() {
        try {
            val display = windowManager.defaultDisplay
            val best = display.supportedModes
                .filter { it.physicalWidth == display.mode.physicalWidth &&
                          it.physicalHeight == display.mode.physicalHeight }
                .maxByOrNull { it.refreshRate } ?: return
            // Ask every way the platform understands; some vendors honour only
            // one of mode id / float rate, and HarmonyOS may clamp app requests
            // entirely, in which case we silently stay at 60 Hz.
            val lp = window.attributes
            lp.preferredDisplayModeId = best.modeId
            lp.preferredRefreshRate = best.refreshRate
            window.attributes = lp
        } catch (_: Throwable) { }
    }

    /**
     * Build the floating corner toolbar as two corner icons + one
     * horizontal popup panel.
     *
```

```
     *  - Top-left: a single 36dp "≡" circle icon. Tapping expands a pill
     *  panel to the **right** of the icon (same Y, no second row below)
     *  containing five commands: 加速 · 重开 · 金手 · 存档 · 读档.
     *  - Top-right: a dedicated "✕" exit circle icon, always on, with a
     *  red accent so it reads as "back to library". Hoisted out of the
     *  panel because users frequently want to leave the game and the
     *  exit command is the single most-common transition.
     *  - Tapping the left icon again, tapping the panel background, or
     *  tapping a panel button (which fires its action) collapses the
     *  panel.
     *
     * Geometry is in screen-percentage terms (icons at the top corners,
     *  panel anchored to the icon's right edge) so the layout scales
     * identically on every phone size.
     *
     * @return Triple(menuIcon, exitIcon, panel) — both icons are always
     *   visible; the panel starts hidden.
     */
    private fun buildMenuBar(sizing: Sizing, netplay: Boolean = false): Triple<View, View, View> {
        // Every panel / button / icon dimension is a fraction of the
        // current screen (see [Sizing]). Reference: HBN-AL00 landscape
        // (~640×360dp, 411dp tall playfield). The percentages reproduce
        // the prior 36dp icon / 6dp margin / 4dp gap / 20dp panel radius
        // / 15dp button radius / 52×32dp button / 1dp stroke / 2dp
        // horizontal padding on that device; on any other phone the
        // proportions stay the same so the toolbar still looks correct.

        // ---------- panel + button factories (same look as before) ----------

        fun panel() = LinearLayout(this).apply {
            orientation = LinearLayout.HORIZONTAL
            // No backing plate and no padding: like the virtual pad's
            // expanded action row, the five commands are free-floating
            // individual pills. The opener-to-panel gap (below) compensates
            // for the first child's start margin so spacing matches the
            // pad's 1.2%-of-width inter-pill rhythm exactly.
            setPadding(0, 0, 0, 0)
            background = null
            // Tapping the panel background (outside any button) collapses
            // the panel — gives users a way to dismiss without finding the
            // corner icon again. LinearLayout's setOnClickListener doesn't
            // catch taps that don't land on a child button, so we use a
            // touch listener with explicit hit-testing on the gaps.
            isClickable = true
            isFocusable = true
            setOnTouchListener { v, ev ->
                if (ev.actionMasked == android.view.MotionEvent.ACTION_UP) {
                    val hitBtn = (0 until childCount).any { i ->
                        getChildAt(i).let { c ->
                            ev.x >= c.left && ev.x <= c.right &&
```

```
                        ev.y >= c.top && ev.y <= c.bottom
                    }
                }
                if (!hitBtn) v.visibility = View.GONE
            }
            false
        }
    }

    // All corner controls share ONE visual language with the virtual
    // pad's collapsible pills (VirtualPadView pillW/pillH =
    // ref*0.1988 x ref*0.077, corner radius ref*0.05, translucent
    // white fill + white hairline, amber fill while pressed) so the
    // native toolbar and the touch overlay read as one UI.
    val pillWpx = sizing.sp(0.1988f)
    val pillHpx = sizing.sp(0.077f)

    fun pillDrawable(accent: Int, pressed: Boolean): Drawable =
        GradientDrawable().apply {
            shape = GradientDrawable.RECTANGLE
            // Radius exceeds half the pill height -> fully rounded caps.
            cornerRadius = sizing.sp(0.05f).toFloat()
            if (pressed) {
                setColor(Color.argb(205, 255, 193, 7))
                setStroke(sizing.sp(0.0072f), Color.argb(245, 255, 235, 170))
            } else {
                setColor(Color.argb(56, 255, 255, 255))
                val strokeA = if (accent == Color.WHITE) 150 else 210
                setStroke(sizing.sp(0.0056f),
                          Color.argb(strokeA, Color.red(accent),
                                     Color.green(accent), Color.blue(accent)))
            }
        }

    fun pill(label: String, accent: Int, action: () -> Unit): Button =
        Button(this).apply {
            text = label
            setTextSize(android.util.TypedValue.COMPLEX_UNIT_PX,
                        sizing.sp(0.042f).toFloat())
            setTextColor(android.content.res.ColorStateList(
                arrayOf(intArrayOf(android.R.attr.state_pressed), intArrayOf()),
                intArrayOf(Color.rgb(45, 32, 0), accent)
            ))
            typeface = android.graphics.Typeface.DEFAULT
            minWidth = 0; minimumWidth = 0
            minHeight = 0; minimumHeight = 0
            stateListAnimator = null
            setPadding(0, 0, 0, 0)
            isAllCaps = false
            includeFontPadding = false
```

```
                background = StateListDrawable().apply {
                    addState(intArrayOf(android.R.attr.state_pressed), pillDrawable(accent, true))
                    addState(intArrayOf(), pillDrawable(accent, false))
                }
                setOnClickListener { action() }
            }

        fun LinearLayout.addBtn(v: Button) {
            // Each popped-out command is one pad pill; 0.6% side margins
            // reproduce the overlay's 1.2%-of-width inter-pill gap.
            addView(v, LinearLayout.LayoutParams(pillWpx, pillHpx).apply {
                marginStart = sizing.wp(0.006f); marginEnd = sizing.wp(0.006f)
            })
        }

        // Single panel containing five commands (exit moved to its own
        // top-right corner icon). Labels stay two Chinese characters so
        // all five pills fit the compact button row.
        val panel = panel()
        if (netplay) {
            // Lockstep session: cheats / fast-forward / save slots / restart
            // would all break determinism — only pause negotiation remains.
            panel.addBtn(pill("暂停", Color.WHITE) { toggleNetplayPause() })
        } else {
            speedButton = pill("加速", Color.WHITE) { cycleSpeed() }
            panel.addBtn(speedButton!!)
            panel.addBtn(pill("摇杆", Color.WHITE) { cycleStickMode() })
            panel.addBtn(pill("重开", Color.WHITE) { confirmRestart() })
            panel.addBtn(pill("金手", Color.WHITE) { openCheatsDialog() })
            panel.addBtn(pill("存档", Color.WHITE) { renderer.saveToSlot() })
            panel.addBtn(pill("读档", Color.WHITE) { openLoadDialog() })
        }

        // ---------- collapsed corner controls (always on) ----------

        // Top-left opener: a pill identical to the pad's "菜单" pill.
        val menuIcon = pill("功能", Color.WHITE) {
            panel.visibility = if (panel.visibility == View.VISIBLE) View.GONE else View.VISIBLE
        }
        menuIcon.layoutParams = android.widget.FrameLayout.LayoutParams(pillWpx, pillHpx)

        // Top-right "✕" exit — a compact round button just big enough for
        // the glyph: square pillH x pillH with fully rounded corners, same
        // fill/stroke family as the pills. Red accent keeps the "back to
        // library" affordance distinct from the neutral controls.
        val exitIcon = pill("✕", Color.rgb(0xEF, 0x9A, 0x9A)) { exitToLibrary() }
        exitIcon.layoutParams = android.widget.FrameLayout.LayoutParams(pillHpx, pillHpx)
        exitIcon.contentDescription = "退出到文件库"
        menuIcon.contentDescription = "功能菜单"
```

```
                panel.visibility = View.GONE
        return Triple(menuIcon, exitIcon, panel)
    }

    /** Cycles normal -> x2 -> x4 -> x6 -> x8 -> normal. */
    private fun cycleSpeed() {
        if (handle == 0L) return
        speedIndex = (speedIndex + 1) % speedSteps.size
        val mult = speedSteps[speedIndex]
        NativeBridge.nativeSetSpeed(handle, mult)
        // Honour the "fast-forward audio" setting: when off, pause the
        // audio stream entirely while running at >×1 (otherwise the
        // accelerated audio artefacts are more jarring than silence).
        val audioPaused = mult > 1 && !fastForwardAudioPref
        NativeBridge.nativeSetAudioPaused(handle, audioPaused)
        speedButton?.text = "加速×$mult"
        Toast.makeText(
            this,
            when {
                mult == 1 -> "已恢复常速"
                audioPaused -> "游戏加速 ×$mult（音效已静音以保证稳定性）"
                else -> "游戏加速 ×$mult（音效同步加快）"
            },
            Toast.LENGTH_SHORT
        ).show()
    }

    /**
     * Cycles the virtual pad's stick mode STICK -> DPAD8 -> DPAD4 -> STICK and
     * persists it, mirroring the OHOS function-menu "摇杆" command
     * (EmulatorSpike.onCommand). STICK = analog knob (8-way digital),
     * DPAD8 = low-dead-zone 8-way pad, DPAD4 = 4-way cross. The new mode is
     * applied live (VirtualPadView.StickMode's setter rebuilds + redraws) and
     * written to the shared-prefs key the Flutter SettingsSheet and
     * [applyPersistedSettings] read, so a later launch / settings open reflects
     * it.
     */
    private fun cycleStickMode() {
        if (!::pad.isInitialized) return
        val order = arrayOf(
            VirtualPadView.StickMode.STICK,
            VirtualPadView.StickMode.DPAD8,
            VirtualPadView.StickMode.DPAD4
        )
        val idx = (order.indexOf(pad.stickMode) + 1) % order.size
        val next = order[idx]
        pad.stickMode = next
        // Persist so the Flutter SettingsSheet and applyPersistedSettings stay
        // in lockstep with this in-emulator toggle.
        getSharedPreferences("FlutterSharedPreferences", MODE_PRIVATE)
```

```
            .edit().putString("flutter.stickMode", next.name).apply()
        val name = when (next) {
            VirtualPadView.StickMode.STICK -> "模拟摇杆"
            VirtualPadView.StickMode.DPAD8 -> "十字键(八向)"
            VirtualPadView.StickMode.DPAD4 -> "十字键(四向)"
        }
        Toast.makeText(this, "摇杆模式: $name", Toast.LENGTH_SHORT).show()
    }

    // ------------------------------------------------------------ cheats UI

    // ------------------------------------------------- save slots / load UI

    /** Automatic background snapshot; distinct CRC keys isolate games. */
    private fun stateFile() = File(filesDir, "saves/$saveKey.state")

    /** Manual save slots: up to three, rotated oldest-first. */
    private fun slotStateFile(i: Int) = File(filesDir, "saves/$saveKey.slot$i.state")
    private fun slotThumbFile(i: Int) = File(filesDir, "saves/$saveKey.slot$i.jpg")

    private data class Slot(val index: Int, val state: File, val thumb: File, val time: Long)

    /**
     * Existing slots, newest first. Enumerates the saves directory once and
     * matches the saveKey prefix (rather than probing three fixed paths), so
     * the picker reflects whatever is actually on disk.
     */
    private fun listSlots(): List<Slot> {
        val dir = File(filesDir, "saves")
        val kids = dir.listFiles() ?: return emptyList()
        val pattern = Regex(
            "^" + Regex.escape(saveKey) + "\\.slot(\\d+)\\.state$")
        return kids.mapNotNull { f ->
            val i = pattern.find(f.name)?.groupValues?.get(1)?.toIntOrNull()
                ?: return@mapNotNull null
            if (i !in 0 until SLOT_COUNT) return@mapNotNull null
            Slot(i, f, slotThumbFile(i), f.lastModified())
        }.sortedByDescending { it.time }
    }

    /**
     * Slot to write next: the first free index, otherwise the oldest of the
     * three (so the newest three saves are always kept).
     */
    private fun nextSlotIndex(): Int {
        val existing = listSlots()
        if (existing.size < SLOT_COUNT) {
            val used = existing.map { it.index }.toSet()
            return (0 until SLOT_COUNT).first { it !in used }
        }
```

```
                return existing.minByOrNull { it.time }!!.index
    }

    private fun fmtSlotTime(t: Long): String {
        val hm = SimpleDateFormat("HH:mm", Locale.getDefault())
        val md = SimpleDateFormat("M月d日", Locale.getDefault())
        val now = Calendar.getInstance()
        val then = Calendar.getInstance().apply { timeInMillis = t }
        val sameDay = now.get(Calendar.YEAR) == then.get(Calendar.YEAR) &&
            now.get(Calendar.DAY_OF_YEAR) == then.get(Calendar.DAY_OF_YEAR)
        val yest = (now.get(Calendar.YEAR) == then.get(Calendar.YEAR)) &&
            now.get(Calendar.DAY_OF_YEAR) - then.get(Calendar.DAY_OF_YEAR) == 1
        return when {
            sameDay -> "今天 ${hm.format(t)}"
            yest -> "昨天 ${hm.format(t)}"
            else -> "${md.format(t)} ${hm.format(t)}"
        }
    }

    /**
     * Deterministic light dialog context. The activity runs a legacy
     * Theme.Black/Theme.Light theme (dark window for the emulator), and EMUI
     * renders AlertDialog surfaces light while views inflated with the activity
     * context keep a white-ish textColorPrimary in dark mode — producing
     * invisible labels. Building both the dialog and its custom content views
     * from this context guarantees dark text on the light Material dialog.
     */
    private val dialogCtx: android.content.Context by lazy {
        android.view.ContextThemeWrapper(
            this, android.R.style.Theme_Material_Light_Dialog_Alert)
    }
    private fun dialogBuilder(): AlertDialog.Builder =
        AlertDialog.Builder(dialogCtx, android.R.style.Theme_Material_Light_Dialog_Alert)

    /** Load picker: thumbnail + time per slot, tap to load, long row to delete. */
    private fun openLoadDialog() {
        // Rescan on a background thread immediately before showing the
        // picker. Right after a cold launch the very first probe used to
        // race disk settling and wrongly reported "no manual saves"; scan
        // the live directory (rechecking once if empty) so that message
        // only appears when no .slotN.state file is really there.
        val holder = FrameLayout(dialogCtx).apply {
            val pb = android.widget.ProgressBar(dialogCtx)
            addView(pb, FrameLayout.LayoutParams(
                ViewGroup.LayoutParams.WRAP_CONTENT,
                ViewGroup.LayoutParams.WRAP_CONTENT,
                Gravity.CENTER))
            setPadding(0, sizing.hp(0.03f).toInt(), 0, sizing.hp(0.03f).toInt())
        }
        val loading = dialogBuilder()
```

```
                .setTitle("读取存档")
                .setView(holder)
                .setNegativeButton("取消", null)
                .create()
            loading.show()
            Thread {
                var slots = listSlots()
                if (slots.isEmpty()) {
                    try { Thread.sleep(300) } catch (_: InterruptedException) {}
                    slots = listSlots()
                }
                val snapshot = slots
                runOnUiThread {
                    if (isFinishing || isDestroyed) return@runOnUiThread
                    loading.dismiss()
                    renderLoadDialog(snapshot)
                }
            }.start()
        }

        private fun renderLoadDialog(slots: List<Slot>) {
            val ctx = dialogCtx
            if (slots.isEmpty()) {
                dialogBuilder()
                    .setTitle("读取存档")
                    .setMessage("还没有手动存档。\n游戏中点菜单「存档」即可保存当前进度，最近 3 个存档会自动保留。")
                    .setPositiveButton("知道了", null)
                    .show()
                return
            }

            // All dialog dimensions are screen-relative percentages of the
            // activity's captured `sizing` (see [Sizing]). Reference: HBN-AL00
            // 360dp-wide landscape; percentages reproduce the prior dp values
            // on that device and scale with the screen elsewhere.
            val list = LinearLayout(ctx).apply {
                orientation = LinearLayout.VERTICAL
                setPadding(
                    sizing.wp(0.022f), sizing.hp(0.011f),
                    sizing.wp(0.022f), sizing.hp(0.011f)
                )
            }
            val scroll = ScrollView(ctx).apply { addView(list) }

            val dialog = dialogBuilder()
                .setTitle("读取存档（${slots.size}/$SLOT_COUNT）")
                .setView(scroll)
                .setNegativeButton("取消", null)
                .create()
```

```
            slots.forEachIndexed { ordinal, slot ->
                val row = LinearLayout(ctx).apply {
                    orientation = LinearLayout.HORIZONTAL
                    gravity = Gravity.CENTER_VERTICAL
                    setPadding(
                        sizing.wp(0.028f), sizing.hp(0.022f),
                        sizing.wp(0.028f), sizing.hp(0.022f)
                    )
                    isClickable = true
                    val outValue = android.util.TypedValue()
                    context.theme.resolveAttribute(
                        android.R.attr.selectableItemBackground, outValue, true)
                    setBackgroundResource(outValue.resourceId)
                }

                val thumb = ImageView(ctx).apply {
                    scaleType = ImageView.ScaleType.CENTER_CROP
                    if (slot.thumb.exists()) setImageBitmap(BitmapFactory.decodeFile(slot.thumb.path))
                    val radius = sizing.sp(0.0167f).toFloat()
                    outlineProvider = object : ViewOutlineProvider() {
                        override fun getOutline(view: View, outline: android.graphics.Outline) {
                            outline.setRoundRect(0, 0, view.width, view.height, radius)
                        }
                    }
                    clipToOutline = true
                    background = GradientDrawable().apply {
                        cornerRadius = radius
                        setColor(Color.rgb(24, 27, 38))
                    }
                }
                row.addView(thumb, LinearLayout.LayoutParams(
                    sizing.wp(0.267f), sizing.hp(0.20f)
                ).apply {
                    marginEnd = sizing.wp(0.039f)
                })

                val texts = LinearLayout(ctx).apply {
                    orientation = LinearLayout.VERTICAL
                }
                texts.addView(TextView(ctx).apply {
                    text = "存档 ${ordinal + 1} · 槽位 ${slot.index + 1}"
                    setTextColor(Color.rgb(0x1F, 0x23, 0x29))
                    textSize = 15f
                    typeface = android.graphics.Typeface.DEFAULT_BOLD
                })
                texts.addView(TextView(ctx).apply {
                    text = fmtSlotTime(slot.time)
                    setTextColor(Color.rgb(0x5F, 0x66, 0x70))
                    textSize = 12.5f
                    setPadding(0, sizing.hp(0.0083f), 0, 0)
```

```
                })
            row.addView(texts, LinearLayout.LayoutParams(0,
                ViewGroup.LayoutParams.WRAP_CONTENT, 1f))

            val del = TextView(ctx).apply {
                text = "删除"
                setTextColor(Color.rgb(0xC6, 0x28, 0x28))
                textSize = 13.5f
                setPadding(
                    sizing.wp(0.039f), sizing.hp(0.028f),
                    sizing.wp(0.0167f), sizing.hp(0.028f)
                )
                isClickable = true
                val outValue = android.util.TypedValue()
                context.theme.resolveAttribute(
                    android.R.attr.selectableItemBackground, outValue, true)
                setBackgroundResource(outValue.resourceId)
                setOnClickListener {
                    dialogBuilder()
                        .setTitle("删除该存档？")
                        .setMessage(fmtSlotTime(slot.time) + " 的存档将被删除，此操作不可撤销。")
                        .setNegativeButton("取消", null)
                        .setPositiveButton("删除") { _, _ ->
                            slot.state.delete()
                            slot.thumb.delete()
                            dialog.dismiss()
                            openLoadDialog()
                        }
                        .show()
                }
            }
            row.addView(del)

            row.setOnClickListener {
                dialog.dismiss()
                renderer.loadSlot(slot.state)
            }
            list.addView(row)

            if (ordinal != slots.lastIndex) {
                list.addView(View(ctx).apply {
                    setBackgroundColor(Color.argb(30, 0, 0, 0))
                }, LinearLayout.LayoutParams(ViewGroup.LayoutParams.MATCH_PARENT, 1).apply {
                    marginStart = sizing.wp(0.033f); marginEnd = sizing.wp(0.033f)
                })
            }
        }

        dialog.show()
        dialog.window?.setLayout(
```

```
      this.showToast('读档失败')
    }
    this.loadDialog = false
  }

  private deleteSave(slot: number): void {
    const savesDir = (getSavesDir() as string)
    const crc = (getRomCrc() as string)
    if (savesDir) {
      try { fs.unlinkSync(savesDir + '/' + crc + '.slot' + slot + '.state') } catch (e) {}
      try { fs.unlinkSync(savesDir + '/' + crc + '.slot' + slot + '.rgba') } catch (e) {}
      try { fs.unlinkSync(savesDir + '/' + crc + '.slot' + slot + '.meta') } catch (e) {}
      try { fs.unlinkSync(savesDir + '/' + crc + '.slot' + slot + '.jpg') } catch (e) {}
    }
    this.openLoadDialog() // refresh
  }

  // Android parity: every manual save carries a JPEG thumbnail
  // (saves/<saveKey>.slotN.jpg, quality 85 — grabThumbnail + compress(JPEG)).
  // The native SaveStateSlot emits the raw down-scaled .rgba plus a .meta
  // ("w h"); convert it to a real JPEG here so the on-disk layout matches
  // Android and the load picker can decode it like BitmapFactory does.
  private writeSlotJpg(slot: number): void {
    try {
      const savesDir = (getSavesDir() as string)
      const crc = (getRomCrc() as string)
      if (!savesDir) {
        return
      }
      const base = savesDir + '/' + crc + '.slot' + slot
      const mf: fs.File = fs.openSync(base + '.meta', fs.OpenMode.READ_ONLY)
      const ms = fs.statSync(base + '.meta')
      const mb: ArrayBuffer = new ArrayBuffer(ms.size)
      fs.readSync(mf.fd, mb)
      fs.closeSync(mf.fd)
      const parts = String.fromCharCode(...new Uint8Array(mb)).trim().split(/\s+/)
      const w = Number(parts[0])
      const h = Number(parts[1])
      if (!(w > 0 && h > 0)) {
        return
      }
      const rf: fs.File = fs.openSync(base + '.rgba', fs.OpenMode.READ_ONLY)
      const rs = fs.statSync(base + '.rgba')
      const buf: ArrayBuffer = new ArrayBuffer(rs.size)
      fs.readSync(rf.fd, buf)
      fs.closeSync(rf.fd)
      const opts: image.InitializationOptions = {
        pixelFormat: image.PixelMapFormat.RGBA_8888,
        size: { width: w, height: h }
```

```
      }
      image.createPixelMap(buf, opts).then((pm: image.PixelMap) => {
        const packer = image.createImagePacker()
        const popts: image.PackingOption = { format: 'image/jpeg', quality: 85 }
        packer.packToData(pm, popts).then((data: ArrayBuffer) => {
          const jf = fs.openSync(base + '.jpg',
            fs.OpenMode.READ_WRITE | fs.OpenMode.CREATE | fs.OpenMode.TRUNC)
          fs.writeSync(jf.fd, data)
          fs.closeSync(jf.fd)
          try { packer.release() } catch (e) { }
        }).catch(() => { try { packer.release() } catch (e) { } })
      }).catch(() => { })
    } catch (e) { /* thumbnail is optional */ }
  }

  // EmulatorActivity.fmtSlotTime(): 今天/昨天 HH:mm, else M月d日 HH:mm.
  private fmtSlotTime(secs: number): string {
    const d = new Date(secs * 1000)
    const pad = (n: number): string => (n < 10 ? '0' + n : '' + n)
    const hm = pad(d.getHours()) + ':' + pad(d.getMinutes())
    const startOfDay = (t: Date): number =>
      new Date(t.getFullYear(), t.getMonth(), t.getDate()).getTime()
    const diffDays = Math.round((startOfDay(new Date()) - startOfDay(d)) / 86400000)
    if (diffDays === 0) {
      return '今天 ' + hm
    }
    if (diffDays === 1) {
      return '昨天 ' + hm
    }
    return (d.getMonth() + 1) + '月' + d.getDate() + '日 ' + hm
  }

  // EmulatorActivity delete flow: confirm first, then delete and reopen the
  // picker (deleteSave already refreshes via openLoadDialog).
  private confirmDeleteSave(slot: number, timeSecs: number): void {
    AlertDialog.show({
      title: '删除该存档？',
      message: this.fmtSlotTime(timeSecs) + ' 的存档将被删除，此操作不可撤销。',
      primaryButton: {
        value: '取消',
        action: () => {
        }
      },
      secondaryButton: {
        value: '删除',
        fontColor: '#C62828',
        action: () => {
          this.deleteSave(slot)
        }
      }
```

```
    })
  }

  // EmulatorActivity.confirmRestart()
  private confirmRestart(): void {
    AlertDialog.show({
      title: '重新开始',
      message: '将清除自动存档并回到游戏开头。\n手动保存的存档位与游戏内电池存档不受影响。',
      primaryButton: {
        value: '取消',
        action: () => {
        }
      },
      secondaryButton: {
        value: '重新开始',
        fontColor: '#C62828',
        action: () => {
          resetEmu()
          this.panelOpen = false
        }
      }
    })
  }

  // EmulatorActivity.openCheatsDialog(): load cheats.json keyed by the loaded
  // ROM's CRC32, then open the Switch/Choice dialog (Android parity).
  private openCheatsDialog(): void {
    try {
      // Native side resolves cheats.json by the loaded ROM's CRC and returns the
      // concrete CheatInfo[] as a string — ArkTS never touches a dynamic key.
      const raw = getCheats()
      const list = JSON.parse(raw) as CheatInfo[]
      if (!list || list.length === 0) {
        this.showToast('本游戏暂无金手指')
        return
      }
      this.cheatInfos = list
      const rows: CheatRow[] = list.map((c: CheatInfo): CheatRow => {
        const hasChoice = !!(c.options && c.options.length > 0)
        const sel = c.selected ?? 0
        let code = c.code
        const labels: string[] = []
        const codes: string[] = []
        if (hasChoice) {
          for (const o of c.options!) {
            labels.push((o.label && o.label.length > 0) ? o.label! : (o.code ?? ''))
            codes.push(o.code ?? '')
          }
          const opt = c.options![sel]
          code = (opt && opt.code) ? opt.code! : ''
```

```
        }
        return {
          name: c.name,
          enabled: c.enabled,
          hasChoice: hasChoice,
          choiceLabels: labels,
          choiceCodes: codes,
          choiceSel: sel,
          code: code,
          info: c
        }
      })
      this.cheatRows = rows
      this.cheatsOpen = true
      this.applyCheats()
    } catch (e) {
      this.showToast('金手指读取失败')
    }
  }

  // Push the current toggle/choice state into the native cheat interface.
  // When `persist` is true (a user toggle), mirror the state back into
  // cheats.json so it survives a dialog close / app restart (Android parity).
  private applyCheats(persist: boolean = false): void {
    const out: Array<[string, boolean]> = []
    for (const r of this.cheatRows) {
      if (r.enabled && r.code.length > 0) {
        out.push([r.code, true])
      }
    }
    setCheats(JSON.stringify(out))
    if (persist) this.persistCheats()
  }

  // Write the current toggle/choice state into <cfgDir>/library/cheats.json under
  // the loaded ROM's CRC key. Local store wins over cloud_cheats.json in GetCheats,
  // so reopening the dialog (and seeding at next launch) reflects the saved state.
  private persistCheats(): void {
    try {
      const crc = getRomCrc() as string
      if (!crc || crc === '00000000') return
      const path = this.configDir() + '/library/cheats.json'
      let byCrc: Record<string, CheatInfo[]> = {}
      if (fs.accessSync(path)) {
        const obj = JSON.parse(fs.readTextSync(path)) as CheatStore
        if (obj && obj.byCrc) byCrc = obj.byCrc
      }
      for (const r of this.cheatRows) {
        if (!r.info) continue
        r.info.enabled = r.enabled
```

```
      if (r.hasChoice) r.info.selected = r.choiceSel
    }
    byCrc[crc] = this.cheatInfos
    const file: fs.File = fs.openSync(path, fs.OpenMode.READ_WRITE | fs.OpenMode.CREATE | fs.OpenMode.TRUNC)
    fs.writeSync(file.fd, JSON.stringify({ version: 1, byCrc } as CheatStore))
    fs.closeSync(file.fd)
    console.info(`[EmulatorSpike] persistCheats crc=${crc} n=${this.cheatInfos.length}`)
  } catch (e) {
    console.warn('[EmulatorSpike] persistCheats failed: ' + (e as Error).message)
  }
}

// Android parity: re-apply the persisted/enabled cheats right after the core
// loads, so a previously-enabled cheat is active without re-opening the dialog.
private seedCheats(): void {
  try {
    const raw = getCheats() as string
    const list = JSON.parse(raw) as CheatInfo[]
    if (!list || list.length === 0) return
    const out: Array<[string, boolean]> = []
    for (const c of list) {
      if (c.enabled && c.code && c.code.length > 0) {
        out.push([c.code, true])
      } else if (c.options && c.options.length > 0 && (c.selected ?? -1) >= 0) {
        const opt = c.options[c.selected!]
        if (opt && opt.code && opt.code.length > 0) out.push([opt.code, true])
      }
    }
    if (out.length > 0) setCheats(JSON.stringify(out))
  } catch (e) {
    console.warn('[EmulatorSpike] seedCheats failed: ' + (e as Error).message)
  }
}

// The core finishes loading on a render thread, so wait for it to be live
// (mirrors the netplay bootstrap gate) before seeding.
private seedCheatsWhenReady(deadline: number): void {
  let ready = false
  try { ready = isCoreReady() as boolean } catch (e) { ready = false }
  if (ready) {
    this.seedCheats()
    return
  }
  if (Date.now() > deadline) return
  setTimeout((): void => { this.seedCheatsWhenReady(deadline) }, 100)
}

/// Mirror of seedCheatsWhenReady: once the core is live, auto-load the most
/// recent save state for this ROM (当 记住上次进度 is on). The persisted slot is
/// ROM-specific via crc, so a slot from a different game is never loaded.
```

```
  private seedSaveWhenReady(deadline: number): void {
    if (!this.rememberLastGame || this.resumeSlot < 0 || !this.resumeCrc) {
      return
    }
    let ready = false
    try { ready = isCoreReady() as boolean } catch (e) { ready = false }
    if (ready) {
      const crc = getRomCrc() as string
      if (crc === this.resumeCrc) {
        if (loadStateSlot(this.resumeSlot)) {
          this.showToast('已恢复上次存档 槽 ' + (this.resumeSlot + 1))
        } else {
          this.showToast('上次存档恢复失败')
        }
      }
      return
    }
    if (Date.now() > deadline) return
    setTimeout((): void => { this.seedSaveWhenReady(deadline) }, 100)
  }

  /// Persist the just-written save slot (per-ROM crc) so the next launch of the
  /// same game can resume it. When 记住上次进度 is off, drop the file instead.
  private persistLastSave(slot: number): void {
    if (!this.rememberLastGame) {
      this.clearLastSaveFile()
      return
    }
    const dir = (this.cfgDir && this.cfgDir.length > 0) ? this.cfgDir : (getFilesDir() as string)
    const path = dir + '/last_slot.json'
    try {
      const crc = getRomCrc() as string
      const file: fs.File = fs.openSync(path, fs.OpenMode.READ_WRITE | fs.OpenMode.CREATE | fs.OpenMode.TRUNC)
      fs.writeSync(file.fd, JSON.stringify({ crc: crc, slot: slot, time: Date.now() }))
      fs.closeSync(file.fd)
      console.info(`[EmulatorSpike] persistLastSave crc=${crc} slot=${slot}`)
    } catch (e) {
      console.warn('[EmulatorSpike] persistLastSave failed: ' + (e as ESObject)?.message)
    }
  }

  private clearLastSaveFile(): void {
    const dir = (this.cfgDir && this.cfgDir.length > 0) ? this.cfgDir : (getFilesDir() as string)
    const path = dir + '/last_slot.json'
    try { fs.unlinkSync(path) } catch (e) { /* not present */ }
  }

  @Builder
  cheatsDialogView() {
    Column() {
```

```
      Text('金手指')
        .fontSize((this.ref() * 0.05) + 'px')
        .fontColor('#222222')
        .fontWeight(FontWeight.Bold)
        .alignSelf(ItemAlign.Start)
        .margin({ bottom: this.ref() * 0.016 })
      if (this.cheatRows.length === 0) {
        Text('本游戏暂无金手指')
          .fontColor('#888888')
          .fontSize(14)
          .margin({ bottom: this.ref() * 0.01 })
      }
      // Scroll keeps the list bounded so the 关闭 button (footer below) is always
      // reachable even when a ROM has dozens of cheats.
      Scroll() {
        Column() {
          ForEach(this.cheatRows, (r: CheatRow, i: number) => {
            Row() {
              Column() {
                Text(r.name)
                  .fontColor('#222222')
                  .fontSize(15)
                  .maxLines(2)
              }
              .layoutWeight(1)
              .alignSelf(ItemAlign.Center)
              if (r.hasChoice) {
                Row() {
                  ForEach(r.choiceLabels, (lab: string, oi: number) => {
                    Text(lab)
                      .fontSize(12)
                      .fontColor(r.choiceSel === oi ? C_TEXT_ACTIVE : C_TEXT)
                      .backgroundColor(r.choiceSel === oi ? C_FILL_ACTIVE : C_FILL)
                      .borderRadius(this.ref() * 0.02)
                      .padding({ left: (this.ref() * 0.012) + 'px', right: (this.ref() * 0.012) + 'px',
                        top: (this.ref() * 0.006) + 'px', bottom: (this.ref() * 0.006) + 'px' })
                      .margin({ right: (this.ref() * 0.008) + 'px' })
                      .onClick(() => {
                        this.cheatRows[i].choiceSel = oi
                        this.cheatRows[i].code = this.cheatRows[i].choiceCodes[oi] ?? ''
                        this.cheatRows = this.cheatRows.slice()
                        this.applyCheats(true)
                      })
                  }, (lab: string, oi: number) => 'opt' + i + '_' + oi)
                }
                .layoutWeight(1)
              }
              Toggle({ type: ToggleType.Switch, isOn: r.enabled })
                .onChange((on: boolean) => {
                  this.cheatRows[i].enabled = on
```

```
                this.cheatRows = this.cheatRows.slice()
                this.applyCheats(true)
              })
            }
            .width('100%')
            .padding(this.ref() * 0.01)
            .margin({ bottom: this.ref() * 0.01 })
            .backgroundColor('rgba(255,255,255,0.92)')
            .borderRadius(this.ref() * 0.02)
          }, (r: CheatRow, i: number) => 'cheat' + i)
        }
        .width('100%')
      }
      .layoutWeight(1)
      .width('100%')
      .margin({ top: this.ref() * 0.004, bottom: this.ref() * 0.004 })
      Button('关闭')
        .width('100%')
        .fontSize(15)
        .backgroundColor(C_FILL)
        .fontColor(C_TEXT)
        .borderRadius(this.ref() * 0.02)
        .margin({ top: this.ref() * 0.01 })
        .onClick(() => { this.cheatsOpen = false })
    }
    .width('82%')
    .height((this.ref() * 0.82) + 'px')
    .padding(this.ref() * 0.025)
    .backgroundColor('#FFFFFF')
    .borderRadius(this.ref() * 0.03)
    .position({ x: (this.layX + this.winW / 2) + 'px', y: (this.layY + this.winH / 2) + 'px' })
    .translate({ x: '-50%', y: '-50%' })
  }

  private showToast(msg: string): void {
    this.toast = msg
    setTimeout(() => { this.toast = '' }, 1200)
  }
}

interface XComponentLoadEvent {
  value: object
}

// ===== FILE: app/ohos/entry/src/main/ets/pages/Index.ets =====

import common from '@ohos.app.ability.common';
import { FlutterPage } from '@ohos/flutter_ohos'

let storage = LocalStorage.getShared()
```

```
const EVENT_BACK_PRESS = 'EVENT_BACK_PRESS'

@Entry(storage)
@Component
struct Index {
  private context = getContext(this) as common.UIAbilityContext
  @LocalStorageLink('viewId') viewId: string = "";

  build() {
    Column() {
      FlutterPage({ viewId: this.viewId })
    }
  }

  onBackPress(): boolean {
    this.context.eventHub.emit(EVENT_BACK_PRESS)
    return true
  }
}

// ===== FILE: app/ohos/entry/src/main/ets/plugins/GamePouchPlugin.ets =====
import {
  FlutterPlugin,
  FlutterPluginBinding,
  MethodChannel,
  MethodCall,
  MethodResult,
  StandardMethodCodec,
} from '@ohos/flutter_ohos';
import common from '@ohos.app.ability.common';
import picker from '@ohos.file.picker';
import fs from '@ohos.file.fs';
import deviceInfo from '@ohos.deviceInfo';
import bundleManager from '@ohos.bundle.bundleManager';
import statfs from '@ohos.file.statvfs';
import zlib from '@ohos.zlib';
import router from '@ohos.router';
import { setEmulatorTarget, getRomCrc } from 'libgpretro.so';
import type { NetplaySession } from '../netplay_session';

/// Return shapes mirrored from the Dart side (`ImportedRom`, `ImportedPackage`,
/// `ImportedMovesConfig`/`ImportedPadConfig`). ArkTS strict mode forbids inline
/// object types, so they are declared as named interfaces.
interface NameSize {
  name: string;
  size: number;
}

interface RomInfo {
  name: string;
```

```
  originalName: string;
  relativePath: string;
  size: number;
  crc32: number;
  /// True when the file starts with the iNES magic ("NES\x1A"). Mirrors
  /// Android's `importRom()` — Dart currently ignores it, but the field keeps
  /// the two platforms' return shapes identical.
  ines: boolean;
}

interface PkgInfo {
  path: string;
  name: string;
  size: number;
}

interface JsonInfo {
  path: string;
  name: string;
  size: number;
  ok: boolean;
}

/// One file staged into the sandbox by [GamePouchPlugin.importFile].
interface StagedFile {
  /// Absolute path of the staged copy.
  path: string;
  /// File name actually used on disk (sanitised / de-duplicated).
  name: string;
  /// File name as reported by the picker — kept for `ImportedRom.originalName`.
  originalName: string;
  size: number;
}

/// Options controlling how [GamePouchPlugin.importFile] names the staged copy.
interface ImportOpts {
  /// `a.zip` -> `a_1.zip` when it already exists (Android's `uniqueName`).
  unique?: boolean;
  /// Prefix prepended to the name (Android's `moves-<ts>-` / `pad-<ts>-`).
  prefix?: string;
  /// Append `.json` when the picked name has no (or another) extension.
  ensureJsonExt?: boolean;
  /// Overwrite an existing file with the same name instead of de-duplicating
  /// (Android's `importBios` deletes the previous copy first).
  overwrite?: boolean;
}

interface VersionInfo {
  versionCode: number;
  versionName: string;
```

```
}

/// Arguments passed by the Dart side to the storage/emulator/updater methods.
/// Only a small, well-known set of keys is used, so a flat optional interface
/// keeps ArkTS strict mode happy (no `any`/`unknown`).
interface CallArgs {
  name?: string;
  mime?: string;
  sourcePath?: string;
  suggestedName?: string;
  rom?: string;
  core?: string;
}

interface ResumeArgs {
  rememberLastGame?: boolean;
  slot?: number;
  crc?: string;
}

interface LaunchArgs {
  rom: string;
  core: string;
  platform: string;
  pad?: PadArgs;
  // Netplay session config from the Dart lobby (PlayLobbyPage). Forwarded to
  // EmulatorSpike so it can run the bootstrap + lockstep session (§16.4).
  netplay?: Record<string, Object>;
  // App-level toggles from the Flutter SettingsSheet that the native emulator
  // surface consumes (FPS HUD, high refresh rate, fast-forward audio).
  prefs?: PrefsArgs;
  // Resume: last save slot + ROM crc forwarded to EmulatorSpike so it can
  // auto-load the most recent save state after the core is ready.
  resume?: ResumeArgs;
}

/// Virtual-pad config pushed from the Flutter SettingsSheet (AppSettings) at
/// launch, mirroring Android's `applyPersistedSettings()`. All fields optional
/// so a missing key leaves the existing settings.json value untouched.
interface PadArgs {
  stickMode?: string;
  stickDeadzone?: number;
  useDpadUi?: boolean;
  padScale?: number;
  padOffsetX?: number;
  padOffsetY?: number;
}

/// App-level toggles consumed by the native emulator surface. Mirrors the
/// `AppSettings` bools of the same name.
```

```
interface PrefsArgs {
  showFpsHud?: boolean;
  highRefreshRate?: boolean;
  fastForwardAudio?: boolean;
}

/// Shape written to `<filesDir>/settings.json` (read by EmulatorSpike.loadSettings).
interface SettingsJson {
  stickMode: string;
  padScale: number;
  stickDeadzone: number;
  useDpadUi: boolean;
  padOffsetX: number;
  padOffsetY: number;
  showFpsHud: boolean;
  highRefreshRate: boolean;
  fastForwardAudio: boolean;
}

/// ArkTS/NAPI side of [OhosPal]. Registers the four Flutter MethodChannels
/// (`gs/storage`, `gs/emulator`, `gs/updater`, `gs/netplay`) and answers each
/// method call with the OpenHarmony equivalent of the Android/Kotlin behavior.
///
/// SAF-style file picks use `@ohos.file.picker`; the picked document URI is
/// copied into the app sandbox (`filesDir`) so the pure-Dart library code can
/// read it the same way it reads Android-imported files.
export class GamePouchPlugin implements FlutterPlugin {
  private appCtx?: common.Context;
  private uiCtx?: common.UIAbilityContext;
  /// The Flutter plugin lifecycle (`onAttachedToEngine`) only hands us an
  /// `ApplicationContext`, which `@ohos.file.picker` refuses — it needs the
  /// `UIAbilityContext`. The EntryAbility stashes it here so picker-backed
  /// import/export work regardless of which plugin instance the engine
  /// dispatches method calls to.
  private static abilityCtx?: common.UIAbilityContext;
  static setAbilityContext(ctx: common.UIAbilityContext): void {
    GamePouchPlugin.abilityCtx = ctx;
  }
  private ctxForPicker(): common.UIAbilityContext | undefined {
    return this.uiCtx ?? GamePouchPlugin.abilityCtx;
  }
  private channels: MethodChannel[] = [];
  private netplayCh?: MethodChannel;
  // Singleton handle so EmulatorSpike can fire the reverse netplay callback.
  private static instance?: GamePouchPlugin;
  // NetplaySession reference set by EmulatorSpike on init and cleared on
  // exit. Allows the Dart→ArkTS 'stop' method to trigger a real teardown.
  private static netplayRef?: NetplaySession;

  constructor(uiCtx?: common.UIAbilityContext) {
```

```
    this.uiCtx = uiCtx;
  }

  getUniqueClassName(): string {
    return 'GamePouchPlugin';
  }

  onAttachedToEngine(binding: FlutterPluginBinding): void {
    this.appCtx = binding.getApplicationContext();
    this.register(binding, 'gs/storage');
    this.register(binding, 'gs/emulator');
    this.register(binding, 'gs/updater');
    this.register(binding, 'gs/netplay');
  }

  onDetachedFromEngine(binding: FlutterPluginBinding): void {
    for (const ch of this.channels) {
      ch.setMethodCallHandler(null);
    }
    this.channels = [];
  }

  private register(binding: FlutterPluginBinding, name: string): void {
    const ch = new MethodChannel(binding.getBinaryMessenger(), name, StandardMethodCodec.INSTANCE);
    const self = this;
    ch.setMethodCallHandler({
      onMethodCall: (call: MethodCall, result: MethodResult): void => {
        self.dispatch(name, call, result);
      },
    });
    this.channels.push(ch);
    if (name === 'gs/netplay') {
      // Keep a handle so we can invoke the reverse (native -> Dart) callback
      // `playEnded` used by netplay teardown.
      this.netplayCh = ch;
    }
    GamePouchPlugin.instance = this;
  }

  private filesDir(): string {
    return this.appCtx?.filesDir ?? '';
  }

  private dispatch(channel: string, call: MethodCall, result: MethodResult): void {
    const method: string = call.method;
    if (channel === 'gs/emulator') {
      // The emulator channel receives a single JSON-string argument (a Map
      // argument is not decoded by the OHOS MethodChannel).
      this.onEmulator(method, call.args as string, result);
      return;
```

```
    }
    const args: CallArgs = call.argument as CallArgs;
    const rawArgs: string = call.args as string;
    if (channel === 'gs/storage') {
      this.onStorage(method, args, rawArgs, result);
    } else if (channel === 'gs/updater') {
      this.onUpdater(method, args, rawArgs, result);
    } else if (channel === 'gs/netplay') {
      this.onNetplay(method, args, result);
    } else {
      result.notImplemented();
    }
  }

  /// gs/netplay (Dart -> native). The reverse direction (native -> Dart
  /// `playEnded`) is invoked by [emitPlayEnded] on the very same channel.
  private onNetplay(method: string, args: CallArgs, result: MethodResult): void {
    if (method === 'stop') {
      // Dart asks for a teardown when it observes that the emulator page was
      // dismissed (PeersController treats the Flutter engine's paused ->
      // resumed transition as "the emulator is gone"), because the native ->
      // Dart `playEnded` invoke can be dropped while the ArkUI page still
      // covers the engine. Without this handler the call used to fall through
      // to notImplemented() and the Dart session was never released.
      this.stopNetplaySession();
      result.success(true);
      return;
    }
    result.notImplemented();
  }

  /// Tears down any live netplay session. `NetplaySession.stop()` runs the full
  /// teardown (UDP sockets closed, BYE to the peer, SRAM flushed) and emits the
  /// real `playEnded` with the session's roomId — that event is what frees the
  /// Dart-side single-session slot.
  private stopNetplaySession(): void {
    const n = GamePouchPlugin.netplayRef;
    if (n) {
      try { n.stop() } catch (e) { console.warn('[GamePouchPlugin] netplay.stop() threw: ' + JSON.stringify(e)) }
      return;
    }
    // No live session: still emit, with a room id that matches nothing, so a
    // Dart caller waiting on this channel gets an answer instead of silence.
    this.emitPlayEnded(0, true);
  }

  // ---------------------------------------------------------------------------
  // gs/storage
  // ---------------------------------------------------------------------------
  private async onStorage(method: string, args: CallArgs, rawArgs: string, result: MethodResult): Promise<void> {
```

```
    // OHOS MethodChannel does not decode Dart Map arguments, so the storage
    // methods that need multiple fields receive a JSON *string* (mirroring the
    // gs/emulator channel convention) and we parse it here.
    let obj: ESObject | null = null;
    if (rawArgs && rawArgs.length > 0) {
      try { obj = JSON.parse(rawArgs) as ESObject; } catch (_) { obj = null; }
    }
    const getStr = (k: string): string =>
      (obj !== null && typeof obj[k] === 'string') ? (obj[k] as string) : '';
    switch (method) {
      case 'getFilesDir':
        result.success(this.filesDir());
        return;
      case 'getDeviceModel': {
        // Mirrors Android's `"$MANUFACTURER $MODEL"` so 「发现伙伴」 renders a
        // human-friendly default name (e.g. "HUAWEI Pura70") instead of a bare
        // model code; falls back to "OpenHarmony" like Android's "Android".
        try {
          const model: string = deviceInfo.productModel ?? '';
          const maker: string = deviceInfo.manufacture ?? '';
          const parts: string[] = [];
          if (maker.trim().length > 0) parts.push(maker.trim());
          if (model.trim().length > 0) parts.push(model.trim());
          const joined: string = parts.join(' ').trim();
          result.success(joined.length > 0 ? joined : 'OpenHarmony');
        } catch (e) {
          result.success('OpenHarmony');
        }
        return;
      }
      case 'getAvailableBytes':
        try {
          const free = await statfs.getFreeSize(this.filesDir());
          result.success(Number(free));
        } catch (e) {
          result.success(0);
        }
        return;
      // NOTE: the Dart side calls this `getRunningSaveKey` (see
      // AndroidPal/OhosPal.runningSaveKey) — the case label must match exactly
      // or the call silently drops into `notImplemented()`.
      case 'getRunningSaveKey': {
        // Report the CRC of the ROM currently loaded by the in-process native
        // emulator (gpretro.getRomCrc returns an 8-hex-char string), so peer
        // import can lock saves for the running game — mirroring Android's
        // getRunningSaveKey. Returns null when no core is loaded ("00000000").
        try {
          const crc = getRomCrc() as string;
          const empty = !crc || crc === '00000000' || crc.toUpperCase() === '00000000';
          result.success(empty ? null : crc);
```

```
        } catch (e) {
          result.success(null);
        }
        return;
      }
      case 'acquireMulticastLock':
      case 'releaseMulticastLock':
        // OHOS exposes no public Wi-Fi multicast-lock API. The `INTERNET`
        // permission already permits broadcast/multicast sockets (the peer
        // discovery protocol uses 255.255.255.255 broadcast, not multicast
        // groups), so there is nothing to acquire/release. Report success so the
        // Dart discovery flow proceeds instead of bailing on a false failure.
        result.success(true);
        return;
      case 'deleteBios': {
        const name: string = getStr('name');
        result.success(this.deleteBios(name));
        return;
      }
      case 'importRomViaSaf': {
        const mime: string = getStr('mime') || '*/*';
        this.importFile<RomInfo>('roms', mime, { unique: true },
          (f: StagedFile): RomInfo => {
            const crc = crc32File(f.path);
            return {
              name: f.name,
              originalName: f.originalName,
              relativePath: `roms/${f.name}`,
              size: f.size,
              crc32: crc,
              ines: sniffInes(f.path),
            };
          })
          .then((m: RomInfo | null) => result.success(m))
          .catch((e: Error) => result.error('IMPORT_FAILED', e.message, ''));
        return;
      }
      case 'importPackageViaSaf':
        this.importFile<PkgInfo>('import_cache', '*/*', { unique: true },
          (f: StagedFile): PkgInfo => {
            return { path: `import_cache/${f.name}`, name: f.name, size: f.size };
          })
          .then((m: PkgInfo | null) => result.success(m))
          .catch((e: Error) => result.error('IMPORT_FAILED', e.message, ''));
        return;
      case 'importBiosViaSaf':
        this.importFile<string>('system', '*/*', { overwrite: true },
          (f: StagedFile): string => f.name)
          .then((m: string | null) => result.success(m))
          .catch((e: Error) => result.error('IMPORT_FAILED', e.message, ''));
```

```
                return;
      case 'importBiosBatchViaSaf':
        this.importFiles('system').then((m: string[] | null) => result.success(m)).catch((e: Error) => result.error('IMPORT_FAILED', e.message, ''));
        return;
      // Android names the staged copy `moves-<ts>-<name>.json` and only marks
      // `ok` when the JSON has a `profiles` object; the pad variant accepts any
      // JSON object. Mirror both so the Dart editors see identical results.
      case 'importMovesConfigViaSaf':
        this.importJson('library/imports', 'moves', 'profiles')
          .then((m: JsonInfo | null) => result.success(m))
          .catch((e: Error) => result.error('IMPORT_FAILED', e.message, ''));
        return;
      case 'importPadConfigViaSaf':
        this.importJson('library/imports', 'pad', '')
          .then((m: JsonInfo | null) => result.success(m))
          .catch((e: Error) => result.error('IMPORT_FAILED', e.message, ''));
        return;
      case 'exportBiosZipViaSaf':
        this.exportBiosZip().then((ok: boolean) => result.success(ok)).catch((e: Error) => result.error('EXPORT_FAILED', e.message, ''));
        return;
      case 'exportPackageViaSaf':
      case 'exportMovesConfigViaSaf':
      case 'exportPadConfigViaSaf': {
        const sourcePath: string = getStr('sourcePath');
        const suggestedName: string = getStr('suggestedName') || 'gamepouch_export';
        // Pre-fill the picker's file name (Android passes EXTRA_TITLE) so the
        // user gets `moves-kof97.json` instead of an untitled document.
        this.exportFile(sourcePath, suggestedName).then((ok: boolean) => result.success(ok)).catch((e: Error) => result.error('EXPORT_FAILED', e.message, ''))
        return;
      }
      default:
        result.notImplemented();
    }
  }

  // ---------------------------------------------------------------------------
  // gs/emulator
  // ---------------------------------------------------------------------------
  // Resolve the libretro core id from a ROM file extension (mirrors the
  // Android platform->core map). Used when Dart does not pass an explicit core.
  private coreForRom(rom: string): string {
    const n = rom.toLowerCase()
    if (n.endsWith('.nes') || n.endsWith('.unf') || n.endsWith('.unif') || n.endsWith('.fds')) return 'fceumm'
    if (n.endsWith('.gb') || n.endsWith('.gbc')) return 'gambatte'
    if (n.endsWith('.gba')) return 'mgba'
    if (n.endsWith('.md') || n.endsWith('.gen') || n.endsWith('.sms') || n.endsWith('.gg')) return 'picodrive'
    if (n.endsWith('.a26')) return 'stella2014'
    // Arcade (CPS1/CPS2/NeoGeo) ROMs ship as zips -> the store-flavour MAME
    // core (libgs_mame.so). Mirrors Dart's PlatformKind.mame/neogeo -> 'mame'.
    if (n.endsWith('.zip') || n.endsWith('.7z')) return 'mame'
```

```
    return 'fceumm'
  }

  // Parse the launch argument. Dart sends a single JSON string
  // ({"rom":..,"core":..,"platform":..}); a bare string is treated as a rom path.
  private parseLaunch(argJson: string): LaunchArgs {
    const out: LaunchArgs = { rom: '', core: '', platform: '', pad: undefined }
    if (!argJson) {
      return out
    }
    try {
      const p = JSON.parse(argJson) as LaunchArgs
      if (p) {
        out.rom = (typeof p.rom === 'string') ? p.rom : ''
        out.core = (typeof p.core === 'string') ? p.core : ''
        out.platform = (typeof p.platform === 'string') ? p.platform : ''
        if (typeof p.pad === 'object' && p.pad !== null) {
          out.pad = p.pad as PadArgs
        }
        if (typeof p.netplay === 'object' && p.netplay !== null) {
          out.netplay = p.netplay as Record<string, Object>
        }
        if (typeof p.prefs === 'object' && p.prefs !== null) {
          out.prefs = p.prefs as PrefsArgs
        }
        if (typeof p.resume === 'object' && p.resume !== null) {
          out.resume = p.resume as ResumeArgs
        }
      }
    } catch (e) {
      // Not JSON — treat the raw string as a relative rom path.
      out.rom = argJson
    }
    return out
  }

  private onEmulator(method: string, argJson: string, result: MethodResult): void {
    if (method === 'launch') {
      // M1: open the native emulator surface page with the chosen ROM. The ROM
      // path may be relative to the app files dir (e.g. "roms/yx.nes"); resolve
      // it against filesDir so the native dlopen host receives an absolute path.
      const la = this.parseLaunch(argJson)
      const romRel: string = la.rom || 'roms/' + decodeURIComponent('%E5%8F%A3%E8%A2%8B%E5%A6%96%E6%80%AA%E5%8F%B6%E7%BB%BF.gba');
      const core: string = la.core || this.coreForRom(romRel);
      const platform: string = la.platform
      console.info(`[GamePouchPlugin] onEmulator romRel=${romRel} core=${core} platform=${platform}`);
      let romAbs = romRel;
      if (romRel.length > 0 && !romRel.startsWith('/')) {
        romAbs = `${this.filesDir()}/${romRel}`;
      }
```

```
      console.info(`[GamePouchPlugin] onEmulator.launch rom=${romAbs} core=${core}`);
      try {
        // Mirror Android's applyPersistedSettings(): stamp the Flutter
        // SettingsSheet pad config into settings.json BEFORE the emulator surface
        // is created, so EmulatorSpike.loadSettings() picks it up on appear.
        if (la.pad) {
          this.writePadSettings(la.pad, la.prefs);
        }
        // Tell the native host the target ROM up-front (before the surface is
        // created) so OnSurfaceCreated can dlopen the core and start emulation.
        setEmulatorTarget(romAbs, core, la.netplay !== undefined);
        // [platform] is what drives the control panel (NEOGEO 4-button vs MAME
        // 6-button); it must ride along because several platforms share the
        // 'mame' core id, exactly like Android's intent extra.
        // Netplay: forward the lobby-provided session config (if any) so the
        // emulator surface can run the bootstrap + lockstep session (§16.4).
        const params: Record<string, Object> = {}
        params['rom'] = romAbs
        params['core'] = core
        params['platform'] = platform
        // The NAPI `getFilesDir()` the emulator page uses is NOT the same as
        // this plugin's `appCtx.filesDir` on real devices, which made
        // EmulatorSpike.loadSettings()/loadMoves() read (and saveSettings()
        // write) a settings.json / library/moves.json nobody else touches —
        // i.e. the Settings sheet had no effect on the running emulator. Hand
        // the authoritative config dir over explicitly.
        params['filesDir'] = this.filesDir()
        if (la.resume) {
          params['resumeSlot'] = (typeof la.resume.slot === 'number') ? la.resume.slot : -1
          params['resumeCrc'] = (typeof la.resume.crc === 'string') ? la.resume.crc : ''
          params['rememberLastGame'] = (typeof la.resume.rememberLastGame === 'boolean') ? la.resume.rememberLastGame : true
        } else {
          params['resumeSlot'] = -1
          params['resumeCrc'] = ''
          params['rememberLastGame'] = true
        }
        if (la.netplay) {
          params['netplay'] = la.netplay
          console.info(`[GamePouchPlugin] onEmulator.launch netplay role=${(la.netplay['role'] as number) ?? -1}`);
        }
        router.pushUrl({ url: 'pages/EmulatorSpike', params: params });
        console.info('[GamePouchPlugin] router.pushUrl(EmulatorSpike) ok');
        result.success(true);
      } catch (e) {
        console.error('[GamePouchPlugin] router.pushUrl failed: ' + (e as ESObject)?.message);
        result.success(false);
      }
      return;
    }
    if (method === 'stop') {
```

```
            this.stopNetplaySession();
      result.success(true);
      return;
    }
    result.notImplemented();
  }

  /// Native -> Dart reverse callback for netplay session end (mirrors Android's
  /// `gs/netplay playEnded`). Consumed by [OhosPal._playEnds] on the Dart side.
  private emitPlayEnded(roomId32: number, clean: boolean): void {
    try {
      // Android sends {"roomId": <int32>, "clean": bool} and
      // NetplayPlayEnd.fromMap reads m['roomId'] — sending 'roomId32' made the
      // cast throw, the event was dropped and the local PlaySession was never
      // released, so the *next* invite failed with "已有一个联机邀请或对局在进行".
      // Keep roomId32 as well so any older Dart build still parses.
      this.netplayCh?.invokeMethod('playEnded', { 'roomId': roomId32, 'roomId32': roomId32, 'clean': clean });
    } catch (e) {
      console.warn('[GamePouchPlugin] emitPlayEnded failed: ' + (e as ESObject)?.message);
    }
  }

  /// Static entry point used by the emulator (EmulatorSpike) to fire the reverse
  /// (native -> Dart) netplay `playEnded` callback when a session drops/ends.
  public static emitNetplayPlayEnded(roomId32: number, clean: boolean): void {
    GamePouchPlugin.instance?.emitPlayEnded(roomId32, clean);
  }

  /// EmulatorSpike registers its NetplaySession here so the Dart→ArkTS
  /// `stop` handler can trigger a real teardown (closing UDP sockets,
  /// sending BYE to peer, flushing SRAM). Pass `undefined` on exit.
  public static setNetplayRef(netplay: NetplaySession | undefined): void {
    GamePouchPlugin.netplayRef = netplay;
  }

  // ---------------------------------------------------------------------------
  // gs/updater
  // ---------------------------------------------------------------------------
  private onUpdater(method: string, args: CallArgs, rawArgs: string, result: MethodResult): void {
    if (method === 'getVersion') {
      try {
        const info = bundleManager.getBundleInfoForSelfSync(bundleManager.BundleFlag.GET_BUNDLE_INFO_WITH_APPLICATION);
        const v: VersionInfo = { versionCode: info.versionCode, versionName: info.versionName };
        result.success(v);
      } catch (e) {
        const v: VersionInfo = { versionCode: 0, versionName: '' };
        result.success(v);
      }
      return;
    }
```

```
    if (method === 'installApk') {
      // OHOS installs .hap, not .apk. No-op (returns false).
      result.success(false);
      return;
    }
    if (method === 'openStore') {
      // Best-effort: no equivalent store intent on OHOS.
      result.success(null);
      return;
    }
    result.notImplemented();
  }

  // ---------------------------------------------------------------------
  // SAF helpers
  // ---------------------------------------------------------------------

  /// Mirrors Android's `applyPersistedSettings()`: stamps the Flutter
  /// SettingsSheet pad config into `<filesDir>/settings.json` so the OHOS native
  /// pad (EmulatorSpike.loadSettings) honours it. The Flutter side always sends
  /// all six fields, so we overwrite unconditionally (the in-emulator
  /// stickMode/padScale cycle also writes the same six fields via saveSettings).
  private writePadSettings(pad: PadArgs, prefs?: PrefsArgs): void {
    try {
      const path = `${this.filesDir()}/settings.json`;
      const obj: SettingsJson = {
        stickMode: (typeof pad.stickMode === 'string') ? pad.stickMode : 'DPAD8',
        padScale: (typeof pad.padScale === 'number') ? pad.padScale : 1.0,
        stickDeadzone: (typeof pad.stickDeadzone === 'number') ? pad.stickDeadzone : 0.18,
        useDpadUi: (typeof pad.useDpadUi === 'boolean') ? pad.useDpadUi : false,
        padOffsetX: (typeof pad.padOffsetX === 'number') ? pad.padOffsetX : 0.0,
        padOffsetY: (typeof pad.padOffsetY === 'number') ? pad.padOffsetY : 0.0,
        showFpsHud: (prefs && typeof prefs.showFpsHud === 'boolean') ? prefs.showFpsHud : true,
        highRefreshRate: (prefs && typeof prefs.highRefreshRate === 'boolean') ? prefs.highRefreshRate : true,
        fastForwardAudio: (prefs && typeof prefs.fastForwardAudio === 'boolean') ? prefs.fastForwardAudio : true,
      };
      const file: fs.File = fs.openSync(path, fs.OpenMode.READ_WRITE | fs.OpenMode.CREATE | fs.OpenMode.TRUNC);
      fs.writeSync(file.fd, JSON.stringify(obj));
      fs.closeSync(file.fd);
    } catch (e) {
      console.warn('[GamePouchPlugin] writePadSettings failed: ' + (e as Error)?.message);
    }
  }

  /// `yyyyMMdd_HHmmss` stamp used for exported documents (mirrors Android's
  /// `SimpleDateFormat("yyyyMMdd_HHmmss")`).
  private stamp(): string {
    const d = new Date();
    const p = (n: number): string => (n < 10 ? `0${n}` : `${n}`);
    return `${d.getFullYear()}${p(d.getMonth() + 1)}${p(d.getDate())}` +
```

```
    `_${p(d.getHours())}${p(d.getMinutes())}${p(d.getSeconds())}`;
  }

  private ensureDir(rel: string): string {
    const dir = `${this.filesDir()}/${rel}`;
    try {
      fs.mkdirSync(dir, true);
    } catch (e) {
      // already exists
    }
    return dir;
  }

  private copyUriTo(uri: string, destPath: string): NameSize {
    const st = fs.statSync(uri);
    const src = fs.openSync(uri, fs.OpenMode.READ_ONLY);
    try {
      fs.copyFileSync(src.fd, destPath);
    } finally {
      fs.closeSync(src);
    }
    const ns: NameSize = { name: fileNameFromUri(uri), size: Number(st.size) };
    return ns;
  }

  /// Mirrors Android's `sanitize()`: strips characters that are illegal in a
  /// sandbox file name so a picker-provided display name can never escape the
  /// target directory (or smuggle a path separator into a Chinese ROM name).
  private safeName(raw: string): string {
    const cleaned: string = raw.replace(/[\\/:*?"<>|]+/g, '_').trim();
    return cleaned.length > 0 ? cleaned : 'rom';
  }

  /// Mirrors Android's `uniqueName()`: `a.zip` -> `a_1.zip` when `a.zip`
  /// already exists, so re-importing a ROM never silently clobbers the copy
  /// already in the library (whose saves are keyed by that file's CRC).
  private uniqueName(dir: string, name: string): string {
    if (!fs.accessSync(`${dir}/${name}`)) {
      return name;
    }
    const dot: number = name.lastIndexOf('.');
    const base: string = dot > 0 ? name.substring(0, dot) : name;
    const ext: string = dot > 0 ? name.substring(dot) : '';
    let i = 1;
    while (fs.accessSync(`${dir}/${base}_${i}${ext}`)) {
      i++;
    }
    return `${base}_${i}${ext}`;
  }
```

```
  private applyNameOpts(dir: string, raw: string, opts: ImportOpts): string {
    let n: string = this.safeName(raw);
    if (opts.prefix !== undefined && opts.prefix.length > 0) {
      n = `${opts.prefix}-${Date.now()}-${n}`;
    }
    if (opts.ensureJsonExt === true && !n.toLowerCase().endsWith('.json')) {
      n = `${n}.json`;
    }
    if (opts.unique === true) {
      n = this.uniqueName(dir, n);
    }
    return n;
  }

  private async importFile<T>(relDir: string, mime: string, opts: ImportOpts, build: (f: StagedFile) => T): Promise<T | null> {
    const ctx = this.ctxForPicker();
    if (!ctx) {
      return null;
    }
    const dp = new picker.DocumentViewPicker(ctx);
    const uris: Array<string> = await dp.select({ maxSelectNumber: 1 });
    if (!uris || uris.length === 0) {
      return null;
    }
    const uri = uris[0];
    const dir = this.ensureDir(relDir);
    const tmp = this.copyUriTo(uri, `${dir}/_tmp_import`);
    const finalName: string = this.applyNameOpts(dir, tmp.name, opts);
    const destPath = `${dir}/${finalName}`;
    if (opts.overwrite === true && fs.accessSync(destPath)) {
      fs.unlinkSync(destPath);
    }
    fs.renameSync(`${dir}/_tmp_import`, destPath);
    return build({ path: destPath, name: finalName, originalName: tmp.name, size: tmp.size });
  }

  private async importFiles(relDir: string): Promise<string[] | null> {
    const ctx = this.ctxForPicker();
    if (!ctx) {
      return null;
    }
    const dp = new picker.DocumentViewPicker(ctx);
    const uris: Array<string> = await dp.select({ maxSelectNumber: 100 });
    if (!uris || uris.length === 0) {
      return null;
    }
    const dir = this.ensureDir(relDir);
    const names: string[] = [];
    let idx = 0;
    for (const uri of uris) {
```

```
      const tmp = this.copyUriTo(uri, `${dir}/_tmp_${idx}`);
      // Android's importBios() replaces an existing copy with the same name
      // (BIOS blobs are matched by exact name, so de-duplicating `_1` variants
      // would make the core fail to find them).
      const destPath = `${dir}/${this.safeName(tmp.name)}`;
      if (fs.accessSync(destPath)) {
        fs.unlinkSync(destPath);
      }
      fs.renameSync(`${dir}/_tmp_${idx}`, destPath);
      names.push(this.safeName(tmp.name));
      idx++;
    }
    return names;
  }

  /// Mirrors Android's `importMovesConfig` / `importPadConfig`: stages the
  /// picked JSON under `library/imports/` as `<prefix>-<ts>-<name>.json` and
  /// pre-validates it. [requireKey] is the top-level key the payload must have
  /// (`profiles` for moves, empty for the pad layout which accepts any object).
  private async importJson(relDir: string, prefix: string, requireKey: string): Promise<JsonInfo | null> {
    const ctx = this.ctxForPicker();
    if (!ctx) {
      return null;
    }
    const dp = new picker.DocumentViewPicker(ctx);
    const uris: Array<string> = await dp.select({ maxSelectNumber: 1 });
    if (!uris || uris.length === 0) {
      return null;
    }
    const uri = uris[0];
    const dir = this.ensureDir(relDir);
    const tmp = this.copyUriTo(uri, `${dir}/_tmp_json`);
    const finalName: string = this.applyNameOpts(dir, tmp.name,
      { prefix: prefix, ensureJsonExt: true });
    const destPath = `${dir}/${finalName}`;
    fs.renameSync(`${dir}/_tmp_json`, destPath);
    let ok = false;
    try {
      const parsed: ESObject = JSON.parse(fs.readTextSync(destPath)) as ESObject;
      if (parsed !== null && typeof parsed === 'object' && !Array.isArray(parsed)) {
        ok = requireKey.length === 0 || parsed[requireKey] !== undefined;
      }
    } catch (e) {
      ok = false;
    }
    return { path: destPath, name: finalName, size: tmp.size, ok: ok };
  }

  /// Append a single line to `filesDir/export_cache/_export_trace.txt` so a
  /// device-side export failure is diagnosable even when `hilog` ring-buffer
```

```
                                /// entries have been flushed by unrelated system logs.
  private traceExport(line: string): void {
    try {
      const dir = `${this.filesDir()}/export_cache`;
      try { fs.mkdirSync(dir, true); } catch (_) { /* exists */ }
      const f = fs.openSync(
        `${dir}/_export_trace.txt`,
        fs.OpenMode.READ_WRITE | fs.OpenMode.CREATE | fs.OpenMode.APPEND,
      );
      try {
        fs.writeSync(f.fd, `${new Date().toISOString()}  ${line}\n`);
      } finally {
        fs.closeSync(f);
      }
    } catch (e) {
      console.warn('[GamePouchPlugin] traceExport failed: ' + (e as Error)?.message);
    }
  }

  private async exportFile(sourcePath: string, suggestedName?: string): Promise<boolean> {
    this.traceExport(`=== exportFile src=${sourcePath} suggested=${suggestedName ?? ''}`);
    const ctx = this.ctxForPicker();
    this.traceExport('ctx=' + (ctx ? 'OK' : 'NULL'));
    if (!ctx || !sourcePath || !fs.accessSync(sourcePath)) {
      this.traceExport('early return false (missing ctx/src or no source file)');
      return false;
    }
    const dp = new picker.DocumentViewPicker(ctx);
    const saveOpts: ESObject = {};
    if (suggestedName !== undefined && suggestedName.length > 0) {
      saveOpts['newFileNames'] = [suggestedName];
    }
    this.traceExport('calling dp.save() ...');
    let uris: Array<string> = [];
    try {
      uris = await dp.save(saveOpts);
    } catch (e) {
      const msg = 'dp.save THREW: ' + (e as Error)?.message;
      this.traceExport(msg);
      throw new Error(msg);
    }
    this.traceExport('dp.save uris=' + JSON.stringify(uris));
    if (!uris || uris.length === 0) {
      this.traceExport('dp.save returned empty -> user cancelled / picker not shown');
      return false;
    }
    const destUri = uris[0];
    const src = fs.openSync(sourcePath, fs.OpenMode.READ_ONLY);
    try {
      // Open the picked destination URI explicitly. A raw copyFileSync to a
```

```
      // `file://` URI string is rejected on OpenHarmony, so copy fd→fd instead.
      const dst = fs.openSync(
        destUri,
        fs.OpenMode.WRITE_ONLY | fs.OpenMode.CREATE | fs.OpenMode.TRUNC,
      );
      try {
        fs.copyFileSync(src.fd, dst.fd);
      } finally {
        fs.closeSync(dst);
      }
      this.traceExport('copied OK -> ' + destUri);
    } catch (e) {
      const msg = 'copy failed: ' + (e as Error).message;
      this.traceExport(msg);
      throw new Error(msg);
    } finally {
      fs.closeSync(src);
    }
    this.traceExport('exportFile SUCCESS');
    return true;
  }

  private deleteBios(name: string): boolean {
    if (!name) {
      return false;
    }
    const base = `${this.filesDir()}/system`;
    let removed = false;
    const target = `${base}/${name}`;
    if (fs.accessSync(target)) {
      fs.unlinkSync(target);
      removed = true;
    }
    // Also remove any per-core mirror (LockedRom step on Android).
    try {
      const subdirs = fs.listFileSync(base);
      for (const sd of subdirs) {
        const mirrored = `${base}/${sd}/${name}`;
        if (fs.accessSync(mirrored)) {
          fs.unlinkSync(mirrored);
          removed = true;
        }
      }
    } catch (e) {
      // ignore
    }
    return removed;
  }

  /// Zips `<filesDir>/system/` into a single archive via SAF so the user can
```

```
  /// back up their BIOS set. Mirrors Android `exportBiosZipViaSaf`.
  private async exportBiosZip(): Promise<boolean> {
    const ctx = this.ctxForPicker();
    if (!ctx) {
      return false;
    }
    const systemDir = `${this.filesDir()}/system`;
    if (!fs.accessSync(systemDir)) {
      return false;
    }
    const cacheDir = this.ensureDir('import_cache');
    const tmpZip = `${cacheDir}/bios_backup.zip`;
    try {
      if (fs.accessSync(tmpZip)) {
        fs.unlinkSync(tmpZip);
      }
    } catch (e) {
      // ignore
    }
    try {
      await zlib.zipFile(systemDir, tmpZip, {});
    } catch (e) {
      return false;
    }
    const dp = new picker.DocumentViewPicker(ctx);
    // Android pre-fills `gamepouch-bios-<stamp>.zip` via EXTRA_TITLE; do the
    // same so the exported backup doesn't land as an unnamed document.
    const stamp: string = this.stamp();
    const uris: Array<string> = await dp.save({ newFileNames: [`gamepouch-bios-${stamp}.zip`] });
    if (!uris || uris.length === 0) {
      return false;
    }
    const destUri = uris[0];
    const src = fs.openSync(tmpZip, fs.OpenMode.READ_ONLY);
    try {
      fs.copyFileSync(src.fd, destUri);
    } finally {
      fs.closeSync(src);
    }
    try {
      fs.unlinkSync(tmpZip);
    } catch (e) {
      // best-effort cleanup
    }
    return true;
  }
}

// ---------------------------------------------------------------------------
// CRC32 (IEEE 802.3) — used by importRomViaSaf to mirror Android/Kotlin value.
```

```
// ---------------------------------------------------------------------------

/// The `@ohos.file.fileuri` helper is not exported in this SDK version, so we
/// derive the display name from the picked document URI's last path segment.
function fileNameFromUri(uri: string): string {
  let name = uri;
  const slash = uri.lastIndexOf('/');
  if (slash >= 0) {
    name = uri.substring(slash + 1);
  }
  const q = name.indexOf('?');
  if (q >= 0) {
    name = name.substring(0, q);
  }
  const h = name.indexOf('#');
  if (h >= 0) {
    name = name.substring(0, h);
  }
  // SAF document URIs percent-encode non-ASCII file names (e.g. a Chinese ROM
  // comes back as `%E6%8B%B3%E7%9A%87%2097.zip`). Decode so the library shows
  // 拳皇 97.zip instead of the percent-encoded mojibake. A raw UTF-8 segment
  // (no '%') passes through unchanged; an invalid escape keeps the raw string.
  try {
    const decoded = decodeURIComponent(name);
    if (decoded.length > 0) {
      name = decoded;
    }
  } catch (_) {
    // keep raw name if it is not valid percent-encoding
  }
  return name;
}

/// True when [path] starts with the iNES magic (`NES\x1A`). Mirrors the
/// `ines` field Android's `importRom()` reports (that sniff reads "NNN\x1A",
/// which never matches — this uses the real magic). Informational only: Dart's
/// [ImportedRom] ignores it, but the two return shapes stay identical.
function sniffInes(path: string): boolean {
  try {
    const f = fs.openSync(path, fs.OpenMode.READ_ONLY);
    const buf = new ArrayBuffer(4);
    const n = fs.readSync(f.fd, buf);
    fs.closeSync(f);
    if (n < 4) return false;
    const b = new Uint8Array(buf);
    return b[0] === 0x4E && b[1] === 0x45 && b[2] === 0x53 && b[3] === 0x1A;
  } catch (e) {
    return false;
  }
}
```

```
function buildCrcTable(): number[] {
  const table: number[] = [];
  for (let n = 0; n < 256; n++) {
    let c = n;
    for (let k = 0; k < 8; k++) {
      c = (c & 1) ? (0xEDB88320 ^ (c >>> 1)) : (c >>> 1);
    }
    table[n] = c >>> 0;
  }
  return table;
}

const CRC_TABLE: number[] = buildCrcTable();

function crc32File(path: string): number {
  const f = fs.openSync(path, fs.OpenMode.READ_ONLY);
  let crc = 0xFFFFFFFF;
  const chunk = new ArrayBuffer(1024 * 1024);
  try {
    while (true) {
      const n = fs.readSync(f.fd, chunk);
      if (n <= 0) {
        break;
      }
      const view = new Uint8Array(chunk, 0, n);
      for (let i = 0; i < n; i++) {
        crc = (crc >>> 8) ^ CRC_TABLE[(crc ^ view[i]) & 0xFF];
      }
    }
  } finally {
    fs.closeSync(f);
  }
  return (crc ^ 0xFFFFFFFF) >>> 0;
}

// ===== FILE: app/ohos/entry/src/main/ets/plugins/GeneratedPluginRegistrant.ets =====
import { FlutterEngine, Log } from '@ohos/flutter_ohos';

/**
 * Generated file. Do not edit.
 * This file is generated by the Flutter tool based on the
 * plugins that support the Ohos platform.
 */

const TAG = "GeneratedPluginRegistrant";

export class GeneratedPluginRegistrant {

  static registerWith(flutterEngine: FlutterEngine) {
```

```
                try {
    } catch (e) {
      Log.e(
        TAG,
        "Tried to register plugins with FlutterEngine ("
          + flutterEngine
          + ") failed.");
      Log.e(TAG, "Received exception while registering", e);
    }
  }
}
```